

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО «СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
ИНСТИТУТ ПЕРСПЕКТИВНОЙ ИНЖЕНЕРИИ
МЕЖИНСТИТУТСКАЯ БАЗОВАЯ КАФЕДРА

КУРСОВОЙ ПРОЕКТ

по дисциплине

«Программная инженерия»

на тему:

«Разработка мобильного приложения цифрового гардероба»

Выполнил:

Астраков Борис Александрович

студент 3 курса

группы ПИЖ-б-о-23-1

направления подготовки 09.03.04

«Программная инженерия»

направленность (профиль)

«Разработка и сопровождение
программного обеспечения»

очной формы обучения

(подпись)

Руководитель проекта:

Свистунов И.В., доцент

межинститутской базовой кафедры

Работа допущена к защите

(подпись руководителя)

(дата)

Работа выполнена и
защищена с оценкой

Дата защиты

Члены комиссии:

зав. межинститутской
базовой кафедрой

Е. Н. Новикова

(подпись)

доцент МИБК

Ф.В. Самойлов

(подпись)

доцент МИБК

З.М. Альбекова

(подпись)

Ставрополь, 2026 г.

**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»**

УТВЕРЖДАЮ

Зав. межинститутской базовой кафедрой
департамента цифровых, робототехнических
систем и электроники, канд. физ.-мат. наук,
Новикова Е.Н.

« ____ » _____ 2026

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

Кафедра межинститутская базовая

Направление подготовки (специальность) 09.03.04 «Программная инженерия»

Направленность (профиль)/ специализация Разработка и сопровождение программного обеспечения

ЗАДАНИЕ

на курсовой проект

студента

Астраков Борис Александрович

по дисциплине «Программная инженерия»

1. Тема проекта: Разработка мобильного приложения цифрового гардероба

2. Целью выполнения курсового проекта является закрепление и углубление теоретических знаний, полученных в курсе «Программная инженерия», а также формирование практических навыков проектирования, разработки и документирования программного обеспечения на всех этапах жизненного цикла от бизнес-анализа до развертывания готового продукта с использованием архитектурного паттерна PCMEF (Presentation-Control-Mediator-Entity-Foundation).

3. Задачи: провести предпроектное обследование и бизнес-анализ предметной области, разработать модель требований к программному обеспечению, спроектировать архитектуру программной системы на основе паттерна PCMEF, реализовать ядро системы с соблюдением принципов SOLID и слоевой архитектуры, обеспечить качество кода через тестирование и рефакторинг, разработать пользовательский интерфейс в соответствии с выбранной траекторией, обеспечить безопасность и управление транзакциями, предоставить API для интеграции с внешними системами (для соответствующих траекторий), выполнить развертывание системы, подготовить полный комплект проектной документации.

4. Перечень подлежащих разработке вопросов:

а) по теоретической части

1. Провести анализ литературных источников по теме курсового проекта, включая периодику и интернет-ресурсы;
2. Сформулировать цели и задачи разработки;
3. Постановка условия задачи, выбор и описание методов и библиотек для ее решения и технологий, среды программирования.

б) по проектной части

1. Составить алгоритмы для решения поставленных задач;
2. Провести проектирование и моделирование процессов, интерфейса, используя методологии IDEF и UML.

в) по практической части

1. Провести разработку сервера приложения с использованием фреймворка Spring Boot;
2. Провести разработку клиентского приложения с использованием Kotlin;
3. Провести отладку и тестирование разработанного программного продукта;
4. Привести описание файловой и функциональной структур приложения разработки.

5. Исходные данные:

- а) по литературным источникам: ГОСТы, международные стандарты, программные средства, используемые при разработке программного обеспечения;
- б) исходные данные, подготовленные для тестирования объекта профессиональной деятельности (информационной системы/приложения/программного продукта).

6. Список рекомендуемой литературы

1. Мацяшек, Л. Практическая программная инженерия на основе учебного примера / Л. Мацяшек. — М.: Вильямс, 2021.
2. Соммервилл, И. Инженерия программного обеспечения / И. Соммервилл. — 10-е изд. — М.: Вильямс, 2018.
3. Мартин, Р. Чистая архитектура / Р. Мартин. — СПб.: Питер, 2018.
4. Фаулер, М. Шаблоны корпоративных приложений / М. Фаулер. — М.: Вильямс, 2016.

7. Контрольные сроки представления отдельных разделов курсового проекта:

- 25 % - подготовка теоретической части курсового проекта «07» марта 2026 г.
50 % - подготовка проектной части курсового проекта «15» апреля 2026 г.
75 % - подготовка практической части курсового проекта «29» апреля 2026 г.
100 % - оформление и подготовка пояснительной записки курсового проекта «10» мая 2026 г.

8. Срок защиты студентом курсового проекта «23» мая 2026 г.

Дата выдачи задания «14» февраля 2026 г.

Руководитель курсового проекта

доцент межинститутской базовой кафедрой
департамента ЦРСиЭ

И.В. Свистунов

Задание принял к исполнению

студент очной формы обучения
3 курса группы ПИЖ-б-о-23-1

Б.А. Астраков

Содержание

Введение.....	5
1. Аналитическая часть.....	7
1.1. Описание предметной области	7
1.2. Анализ бизнес-процессов (IDEF0)	8
1.3. SWOT-анализ текущего состояния	10
1.4. Анализ аналогов и конкурентов	12
1.5. Обоснование необходимости разработки.....	13
1.6. Экономическое обоснование (ROI).....	14
Выводы	16
2. Проектная часть.....	17
2.1. Модель требований к ПО	17
2.1.1. Use Case диаграмма	17
2.1.2. Спецификация прецедентов.....	18
2.1.3. Глоссарий терминов	18
2.2. Модель предметной области.....	19
2.2.1. Domain Model (диаграмма классов)	19
2.2.2. Описание сущностей и атрибутов.....	20
2.2.3. Бизнес-правила	20
2.3. Архитектурное проектирование	21
2.3.1. Выбор архитектурного стиля.....	21
2.3.2. Диаграмма пакетов (PCMEF)	21
2.3.3. Описание слоев и их ответственности.....	22
2.3.4. Архитектурные решения (ADR).....	22
2.4. Проектирование базы данных	23
2.4.1. ER-диаграмма.....	23
2.4.2. Физическая модель данных	23
2.4.3. DDL-скрипты.....	24
2.5. Детальное проектирование	24
2.5.1. Диаграммы последовательности	24
2.5.2. Диаграмма классов проектирования.....	25
2.5.3. Применение паттернов GoF и архитектурных паттернов	26

Выводы	26
3. Реализационная часть	27
3.1. Реализация бизнес-логики.....	27
3.1.1. Структура проекта	27
3.1.2. Классы-сущности.....	27
3.1.3. Слой доступа к данным	28
3.1.4. Слой управления	28
3.2. Рефакторинг и оптимизация	28
3.2.1. Статический анализ кода	28
3.2.2. Применение паттернов (Data Mapper, Identity Map)	29
3.2.3. Оптимизация запросов	29
3.3. Пользовательский интерфейс	29
3.4. Безопасность и транзакции	30
3.4.1. Аутентификация и авторизация	30
3.4.2. Управление транзакциями	30
3.4.3. Защита от атак	30
3.5. REST API.....	31
3.5.1. Спецификация OpenAPI.....	31
3.5.2. Реализация контроллеров.....	31
3.5.3. Тестирование API	31
Выводы	32
4. Тестирование и обеспечение качества	33
4.1. Модульное тестирование	33
4.2. Интеграционное тестирование	34
4.3. Системное тестирование	34
4.4. Нагрузочное тестирование	35
4.5. Результаты тестирования	35
Выводы	36
5. Развертывание и эксплуатация	37
5.1. Контейнеризация (Docker)	37
5.2. Инструкция по установке	38
5.3. Инструкция по настройке.....	38

5.4. Требования к окружению	39
Выводы	40
6. Управление проектом	41
6.1. WBS (иерархическая структура работ).....	41
6.2. Диаграмма Ганта	42
6.3. Оценка трудозатрат (COCOMO)	43
6.4. Управление рисками	44
Выводы	45
Заключение	46
Список использованных источников	48
Приложение А. Листинги кода	49
Приложение Б. Скриншоты интерфейсов	60
Приложение В. Результаты тестирования	64

Введение

В повседневной жизни пользователи регулярно сталкиваются с трудностями при организации гардероба: забывают о вещах, тратят время на утренний подбор образов и покупают дубли. Существующие решения перегружены социальным и коммерческим функционалом, не поддерживают критичный офлайн-режим и предлагают лишь неупорядоченные коллажи вместо структурированного создания образов. Разработка приложения решает эту проблему и позволяет на практике закрепить навыки проектирования распределенных систем, REST API и кэширования.

Объект исследования — процесс разработки мобильного приложения с клиент-серверной архитектурой и офлайн-режимом. Предмет — методы реализации паттерна PCMEF в связке Kotlin/Android и Java/Spring Boot, а также обеспечение целостности данных при двусторонней синхронизации.

Технологический стек проекта:

- Клиент (Android): Kotlin, Jetpack Compose, MVVM, Room, Retrofit, WorkManager;
- Сервер: Java 17, Spring Boot 3, Spring Data JPA, PostgreSQL, Spring Security (JWT);
- Инструменты: UML, JUnit 5, Mockito, Git, Checkstyle, JaCoCo.

Цель проекта — разработка полнофункционального приложения «Моя полка» для каталогизации одежды и визуального конструктора образов со слотами, реализующего регистрацию, автономную работу и последующую синхронизацию с сервером.

Задачи:

1. Проанализировать предметную область и сформулировать требования к системе в виде пользовательских сценариев;
2. Спроектировать архитектуру на основе адаптированного паттерна PCMEF и модель данных в PostgreSQL;
3. Реализовать серверную часть на Spring Boot с REST API, JWT-аутентификацией и валидацией образов;

4. Разработать Android-клиент на Jetpack Compose с навигацией и обработкой состояний;
5. Обеспечить офлайн-режим через локальное кэширование (Room) и фоновую синхронизацию (WorkManager);
6. Написать модульные и интеграционные тесты (покрытие кода от 40%), внедрить паттерны Data Mapper и Identity Map, провести статический анализ и подготовить документацию.

1. Аналитическая часть

1.1. Описание предметной области

Предметная область данного проекта охватывает сферу персонального управления гардеробом — совокупность процессов, связанных с учетом, систематизацией и подбором предметов одежды и аксессуаров. В современном мире, где средний городской житель владеет десятками и сотнями единиц одежды, задача осознанного использования гардероба становится нетривиальной. Исследования показывают, что люди в среднем используют лишь 20–30% своего гардероба регулярно, тогда как значительная часть вещей остается невостребованной из-за того, что пользователь просто забывает об их существовании или не может быстро подобрать к ним сочетаемые комплекты.

Ключевыми процессами предметной области являются:

- Каталогизация вещей — фиксация атрибутов предмета (категория, сезон, цвет, состояние, место хранения) и его визуального образа (фотография).
- Подбор образов — комбинирование предметов в логически завершенные комплекты с учетом контекста (погода, повод, сезон, личные предпочтения).
- Планирование гардероба — предварительное составление комплектов на будущие дни, анализ частоты использования вещей, выявление «мертвого груза».
- Учет состояния — отслеживание износа, необходимости ремонта или замены предметов.

В последние годы предметная область активно цифровизируется. Если раньше учет гардероба велся вручную (списки в заметках, фотоальбомы), то сегодня на рынке представлено множество мобильных приложений, предлагающих автоматизацию этих процессов. Однако большинство из них ориентированы либо на социальный аспект (публикация образов, рекомендации стилистов), либо на коммерческий (интеграция с магазинами), тогда как

потребность в простом, приватном и работающем офлайн инструменте остается недостаточно удовлетворенной.

С технической точки зрения предметная область требует реализации клиент-серверной архитектуры: мобильное приложение выступает точкой взаимодействия с пользователем, серверная часть обеспечивает хранение данных, аутентификацию и синхронизацию между устройствами, а локальное кэширование гарантирует доступность информации без постоянного подключения к сети.

1.2. Анализ бизнес-процессов (IDEF0)

Для формализации бизнес-процессов системы была построена контекстная диаграмма IDEF0 A-0. Диаграмма отражает четыре типа потоков: входы, выходы, управления и механизмы.

Входы — это данные, которые система преобразует в процессе своей работы:

- Данные учетной записи (email, пароль, имя пользователя, аватар);
- Данные предметов гардероба (фотографии, названия, описания, категории, сезонность);
- Параметры для создания образов (вещи, название и описание образа);
- Команды управления (запросы на поиск, фильтрацию, редактирование, удаление, синхронизацию).

Выходы — результаты функционирования системы:

- Аутентифицированный доступ (JWT-токен, подтверждение входа);
- Каталог вещей (список предметов, результаты поиска и фильтрации);
- Коллекция образов (готовые комплекты, визуализация, рекомендации);
- Персонализированные настройки (тема, язык, параметры профиля);
- Синхронизированные данные (локально сохраненные записи, результаты фоновой синхронизации).

Управления — внешние ограничения и правила, регулирующие работу системы:

- Требования к функционалу (спецификация, пользовательский сценарий);
- Стандарты и протоколы (REST API, OpenAPI/Swagger, JWT, HTTPS);
- Архитектурные ограничения (паттерн PCMEF);
- Технические требования (минимальная версия Android, покрытие тестами, количество экранов и эндпоинтов).
- Механизмы — ресурсы, выполняющие преобразование:
- Мобильное приложение (Kotlin, Jetpack Compose, Retrofit, Room, ViewModel);
- Серверная часть (Java 17+, Spring Boot, Spring Data JPA, Spring Security, PostgreSQL) [10];
- Инфраструктура (REST API сервер, система аутентификации, механизм синхронизации);
- Инструменты разработки (Android Studio, Maven/Gradle, Git [5]).

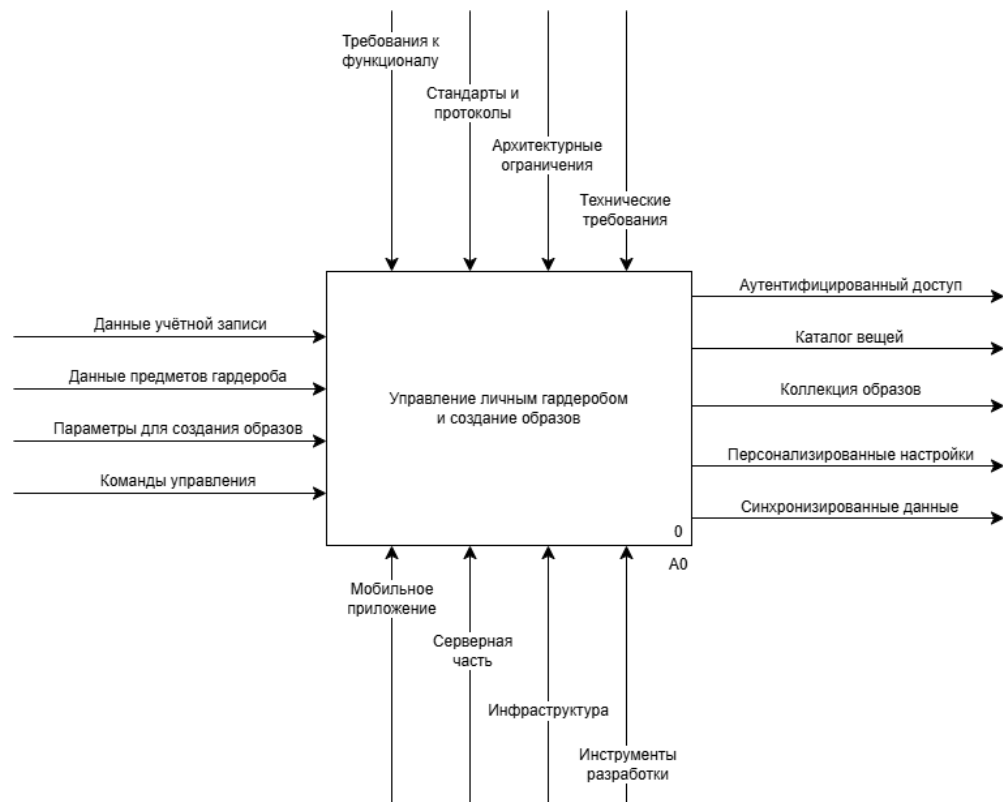


Рисунок 1. Диаграмма IDEF0 A-0

Контекстная диаграмма демонстрирует, что система является замкнутым преобразователем входных данных пользователя в структурированный цифровой гардероб, действуя в рамках заданных архитектурных и технических ограничений с использованием определенного набора технологических механизмов.

1.3. SWOT-анализ текущего состояния

Для оценки перспектив разработки приложения «Моя полка» проведен SWOT-анализ, рассматривающий внутренние сильные и слабые стороны проекта, а также внешние возможности и угрозы.

Сильные стороны (Strengths):

- Четкая архитектура на базе паттерна PCMEF обеспечивает модульность и легкость сопровождения кода [8].
- Поддержка офлайн-режима с локальным кэшированием через Room делает приложение надежным в условиях нестабильного интернет-соединения.
- Структурированный конструктор образов со слотами и валидацией категорий отличает продукт от конкурентов, предлагающих лишь свободные коллажи.
- Использование современных технологий (Kotlin, Jetpack Compose, Spring Boot 3, JWT) гарантирует актуальность решения и его соответствие индустриальным стандартам [14re].
- Открытый исходный код и подробная документация облегчают дальнейшее развитие проекта.

Слабые стороны (Weaknesses):

- Ограниченный функционал MVP по сравнению с зрелыми коммерческими аналогами (отсутствие рекомендаций, интеграции с магазинами, социальных функций).

- Зависимость от одного разработчика создает риск задержек и ограничивает скорость итераций.
- Отсутствие монетизации на этапе курсового проекта не позволяет оценить экономическую устойчивость решения.
- Необходимость ручного ввода данных (фотографий и метаданных вещей) создает барьер входа для новых пользователей.

Возможности (Opportunities):

- Растущий интерес к осознанному потреблению и минимализму формирует устойчивый спрос на инструменты управления гардеробом.
- Развитие технологий машинного обучения открывает перспективы для внедрения функций автоматического подбора образов и анализа стиля.
- Интеграция с погодными API позволит автоматически рекомендовать комплекты в зависимости от прогноза.
- Расширение платформы на iOS и веб-версию увеличит охват аудитории.
- Партнерство с брендами устойчивой моды может стать основой монетизации.

Угрозы (Threats):

- Высокая конкуренция со стороны устоявшихся приложений (Stylebook, Cladwell, Whering) с большими бюджетами на развитие.
- Изменения в политиках Google Play могут потребовать дополнительных усилий по сертификации.
- Зависимость от сторонних сервисов (хостинг, базы данных, API изображений) создает операционные риски.
- Быстрое устаревание технологического стека требует постоянного поддержания актуальности зависимостей.

1.4. Анализ аналогов и конкурентов

На рынке мобильных приложений для управления гардеробом представлено несколько заметных игроков. Их анализ позволяет выявить как лучшие практики, так и пробелы, которые заполняет разрабатываемое решение.

Stylebook (iOS) — одно из старейших приложений в категории. Позволяет загружать фотографии вещей, создавать образы и вести статистику использования. К сильным сторонам относятся богатый функционал и подробная аналитика. К слабым — отсутствие Android-версии, платная модель распространения и устаревший интерфейс. Офлайн-режим поддерживается, но синхронизация между устройствами отсутствует.

Cladwell (iOS, Android) — приложение с акцентом на минимализм и устойчивую моду. Предлагает капсульный гардероб, готовые образы от стилистов и интеграцию с брендами. Сильная сторона — curated-контент и сообщество. Слабая сторона — подписочная модель, ограниченный бесплатный функционал и отсутствие возможности создавать собственные структурированные образы со слотами.

Smart Closet (iOS, Android) — приложение с функциями планирования гардероба, календаря образов и интеграции с погодой. Поддерживает как ручной ввод, так и импорт из интернет-магазинов. Недостатки — перегруженный интерфейс, реклама в бесплатной версии и слабая кастомизация структуры образов.

Whering (iOS, Android) — социальное приложение, позиционируемое как «Shazam для гардероба». Делает акцент на публикации образов и получении обратной связи от сообщества. Сильная сторона — социальный аспект. Слабая — отсутствие глубоких инструментов управления гардеробом и офлайн-режима.

Таблица 1. Сравнительная таблица популярных приложений в категории "цифровой гардероб"

Приложение	Платформа	Офлайн-режим	Синхронизация	Бесплатная модель	Открытый код
Stylebook	IOS	Да	Нет	Нет	Нет
Cladwell	IOS, Android	Частично	Да	Ограничена	Нет
Smart Closet	IOS, Android	Да	Да	Ограничена	Нет
Whering	IOS, Android	Нет	Да	Ограничена	Нет
Моя Полка (текущий проект)	Android	Да	Да		

Анализ показывает, что ни одно из существующих решений не сочетает одновременно структурированный конструктор образов со слотами, полноценный офлайн-режим, открытую архитектуру и бесплатную модель распространения. Именно этот пробел заполняет разрабатываемое приложение.

1.5. Обоснование необходимости разработки

Необходимость разработки приложения «Моя полка» обусловлена совокупностью факторов, которые не находят полного отражения в существующих решениях.

С точки зрения практической необходимости, пользователи Android-устройств лишены доступа к лидеру категории (Stylebook), а альтернативы либо навязывают подписочную модель, либо перегружены социальным функционалом, не относящимся к базовой задаче учета гардероба. Приложение «Моя полка» предлагает сфокусированный на основной задаче инструмент без коммерческих надстроек.

Технически, существующие приложения, как правило, закрыты для модификации и расширения. Разрабатываемое решение с открытым исходным кодом и модульной архитектурой на базе PCMEF создает основу, которую могут развивать другие разработчики — добавлять интеграции, улучшать алгоритмы подбора, адаптировать под специфические нужды.

С образовательной стороны, в рамках курсового проекта разработка позволяет продемонстрировать применение полного цикла современных

практик: от проектирования архитектуры и базы данных до реализации клиент-серверного взаимодействия, локального кэширования, безопасной аутентификации и тестирования. Это формирует компетенции, востребованные в индустрии.

Наконец, концепция структурированных слотов с валидацией категорий (например, невозможность поместить обувь в слот для головного убора) не реализована ни в одном из рассмотренных аналогов в явном виде. Данный подход снижает когнитивную нагрузку на пользователя и предотвращает логические ошибки при составлении образов.

1.6. Экономическое обоснование (ROI)

Хотя проект разрабатывается в учебных целях, оценка его экономической эффективности позволяет продемонстрировать понимание принципов управления ИТ-проектами. Расчет проведен для гипотетического сценария коммерциализации приложения.

Исходные данные для расчета:

- Трудозатраты на разработку MVP: 32,8 человеко-месяца (по оценке СОСОМО II).
- Средняя стоимость человеко-месяца разработчика уровня Middle в РФ: 200 000 рублей.
- Инфраструктурные расходы (хостинг, домен, мониторинг): 15 000 рублей в месяц.
- Планируемый срок окупаемости: 18 месяцев после запуска.

Инвестиции в разработку:

- Разработка: $32,8 \times 200\,000 = 6\,560\,000$ рублей.
- Инфраструктура за период разработки (3 месяца): 45 000 рублей.
- Итого первоначальные инвестиции: 6 605 000 рублей.

Прогнозируемые доходы (пессимистичный сценарий):

- Целевая аудитория: 10 000 активных пользователей через 12 месяцев.

- Конверсия в платную подписку (при введении Premium-функций): 5%.
- Платящих пользователей: 500.
- Стоимость подписки: 149 рублей/месяц.
- Ежемесячный доход: $500 \times 149 = 74\,500$ рублей.
- Годовой доход: 894 000 рублей.

Операционные расходы:

- Инфраструктура: $15\,000 \times 12 = 180\,000$ рублей/год.
- Поддержка и развитие: 200 000 рублей/год.
- Итого: 380 000 рублей/год.

Чистая прибыль за первый год: $894\,000 - 380\,000 = 514\,000$ рублей.

ROI за 18 месяцев:

- Суммарная чистая прибыль: $514\,000 \times 1,5 = 771\,000$ рублей.
- $ROI = (771\,000 - 6\,605\,000) / 6\,605\,000 \times 100\% = -88,3\%$.

Отрицательный ROI в краткосрочной перспективе типичен для мобильных приложений на этапе запуска и объясняется высокими первоначальными инвестициями в разработку. Точка безубыточности достигается примерно через 13 лет при пессимистичном сценарии, что делает модель нежизнеспособной без дополнительных источников дохода.

Альтернативные модели монетизации, способные улучшить ROI:

- Партнерские программы с брендами одежды (комиссия с продаж по реферальным ссылкам);
- Продажа анонимизированных данных о трендах гардероба аналитическим компаниям;
- Freemium-модель с расширенными функциями (AI-рекомендации, экспорт образов, неограниченное количество слотов);
- B2B-направление: корпоративные решения для магазинов одежды и стилистов.

При реализации хотя бы двух из перечисленных моделей срок окупаемости сокращается до 3–4 лет, что соответствует отраслевым показателям для нишевых мобильных приложений.

Выводы

В ходе работы над аналитической частью был проведен анализ предметной области, бизнес-процессов, конкурентной среды и экономических показателей. Результаты анализа подтверждают целесообразность разработки приложения «Моя полка»: существующие решения не закрывают потребность в структурированном, работающем офлайн и открытом инструменте управления гардеробом для Android-пользователей, а выбранная архитектура и технологический стек обеспечивают техническую реализуемость проекта в рамках заданных ограничений.

2. Проектная часть

2.1. Модель требований к ПО

2.1.1. Use Case диаграмма

Для формализации функциональных требований к системе была разработана диаграмма вариантов использования. Основным актором системы выступает «Пользователь». Диаграмма отражает четыре ключевых прецедента верхнего уровня: «Управлять вещами» (UC-005), «Управлять образами» (UC-013), «Настроить приложение» (UC-020) и «Управлять учетной записью» (UC-021).

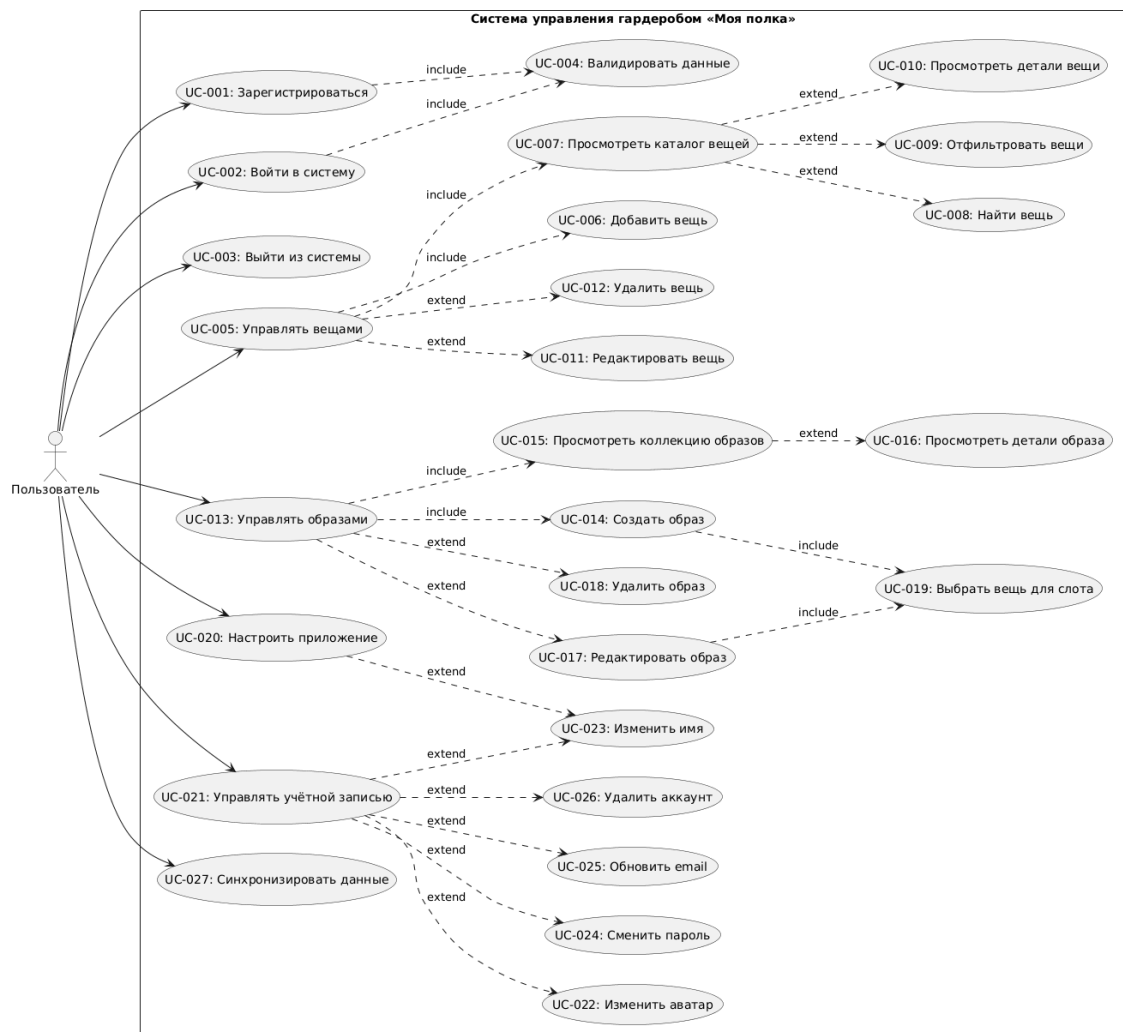


Рисунок 2. Диаграмма вариантов использования

2.1.2. Спецификация прецедентов

Детальное описание основных прецедентов представлено в виде таблицы спецификаций, фиксирующей предусловия, постусловия, основные и альтернативные потоки выполнения, а также сценарии обработки исключений.

UC-005 «Управление вещами» позволяет пользователю добавлять, просматривать, редактировать и удалять предметы гардероба. Ключевым постусловием является сохранение изменений в локальной базе данных (Room) с последующей постановкой в очередь на синхронизацию с сервером. Альтернативный поток включает поиск и фильтрацию по категориям и сезонам.

UC-013 «Управление образами» обеспечивает создание структурированных комплектов одежды через визуальный конструктор со слотами. Исключением является попытка сохранения пустого образа, при которой система блокирует операцию.

UC-020 «Настроить приложение» отвечает за изменение визуальных и функциональных параметров (тема, язык). Изменения применяются к интерфейсу мгновенно.

UC-021 «Управлять учетной записью» включает редактирование профиля, смену пароля и удаление аккаунта. При удалении аккаунта требуется повторная аутентификация и подтверждение потери данных.

2.1.3. Глоссарий терминов

Для обеспечения единообразия понимания терминологии в рамках проекта (Ubiquitous Language) разработан глоссарий. Далее перечислены основные термины.

Вещь (Item) — единица каталога гардероба, обладающая метаданными (название, категория, сезон) и визуальным представлением (фотография).

Образ (Outfit) — структурированный набор вещей, собранный пользователем в единый комплект.

Слот (Slot) — позиционный элемент конструктора образа, жестко привязанный к определенной категории одежды (например, слот «Обувь» принимает только вещи категории «Обувь»).

Синхронизация (Sync) — процесс двустороннего обмена данными между локальным хранилищем устройства и серверной базой данных для обеспечения консистентности.

PCMEF — архитектурный паттерн, разделяющий систему на слои: Presentation, Control, Mediator, Entity, Foundation.

JWT (JSON Web Token) — стандарт передачи утверждений между сторонами, используемый в системе для Stateless-аутентификации.

2.2. Модель предметной области

2.2.1. Domain Model (диаграмма классов)

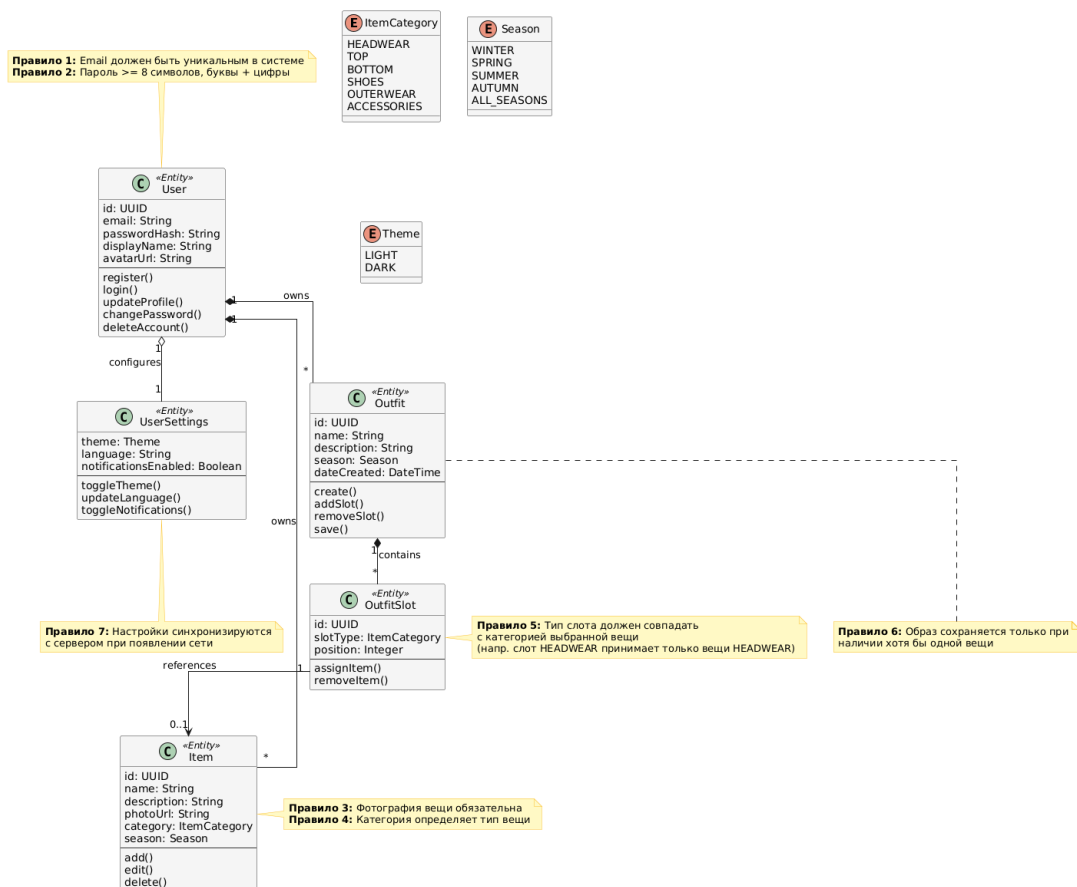


Рисунок 3. Диаграмма классов

Модель предметной области отражает ключевые сущности системы, их атрибуты и взаимосвязи без привязки к конкретной технологии хранения данных. Корневым агрегатом выступает сущность User, владеющая коллекциями Item и Outfit. Связь между образом и вещью реализована не напрямую, а через сущность OutfitSlot, что позволяет хранить контекст использования вещи в рамках конкретного комплекта.

2.2.2. Описание сущностей и атрибутов

User (Пользователь): идентификатор (UUID), email (уникальный), хеш пароля, отображаемое имя, URL аватара.

UserSettings (Настройки): идентификатор пользователя (связь 1:1), тема оформления, язык интерфейса, флаг уведомлений.

Item (Вещь): идентификатор, владелец (User), название, описание, URL изображения, категория (Enum), сезон (Enum), метки времени создания и обновления.

Outfit (Образ): идентификатор, владелец (User), название, описание, сезон, дата создания.

OutfitSlot (Слот образа): идентификатор, ссылка на образ, ссылка на вещь (допускает NULL), тип слота (Enum, совпадает с категорией вещи).

2.2.3. Бизнес-правила

Функционирование системы ограничено следующими бизнес-правилами:

- Уникальность идентификатора: email пользователя должен быть уникальным в рамках всей системы.
- Сложность пароля: пароль должен содержать не менее 8 символов, включая буквы и цифры.
- Консистентность слотов: категория вещи, помещаемой в слот образа, должна строго соответствовать типу слота (например, вещь категории TOP не может быть добавлена в слот типа SHOES).

- Минимальная наполненность образа: образ не может быть сохранен, если в него не добавлена хотя бы одна вещь.
- Целостность при удалении: при удалении вещи из гардероба связанные с ней слоты в образах не удаляются, а обнуляются (становятся NULL), сохраняя структуру шаблона образа.

2.3. Архитектурное проектирование

2.3.1. Выбор архитектурного стиля

Для реализации проекта выбран стиль «Клиент-Сервер» с использованием REST API в качестве контракта взаимодействия [4]. Клиентская часть следует паттерну Model-View-ViewModel, адаптированному под принципы PCMEF [9]: клиент выступает независимым приложением, способным функционировать автономно, а сервер обеспечивает централизованное хранение, безопасность и синхронизацию между устройствами.

2.3.2. Диаграмма пакетов (PCMEF)

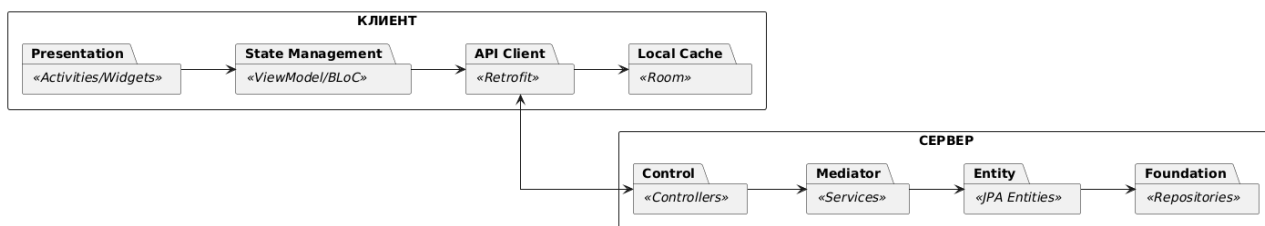


Рисунок 4. Диаграмма пакетов (PCMEF)

Архитектура системы разделена на два независимых контекста (Bounded Contexts): Клиент (Android) и Сервер (Spring Boot). Каждый контекст реализует слои PCMEF.

2.3.3. Описание слоев и их ответственности

Presentation (Представление) отвечает за отображение UI и обработку пользовательского ввода. На клиенте — это Jetpack Compose экраны, на сервере — REST Controllers.

Control (Управление) содержит ViewModel (клиент) и DTO-валидацию (сервер). Управляет состоянием интерфейса и маршрутизацией запросов.

Mediator (Посредник) реализует бизнес-логику и координацию данных. На клиенте — это Repository (решает, откуда брать данные: из сети или кэша), на сервере — Service layer.

Entity (Сущности) – объекты предметной области. JPA Entities на сервере, Room Entities на клиенте.

Foundation (Фундамент) – механизмы доступа к данным. DAO интерфейсы Room на клиенте, JpaRepository на сервере.

2.3.4. Архитектурные решения (ADR)

ADR-1: «Использование Room для локального кэширования». Принято решение использовать Room вместо прямого SQLite API или Realm, так как Room обеспечивает компиляцию SQL-запросов на этапе сборки, нативную поддержку Kotlin Coroutines и Flow, а также простую интеграцию с LiveData/StateFlow.

ADR-2: «Stateless-аутентификация через JWT». Вместо серверных сессий выбран JWT, что позволяет легко масштабировать серверную часть и упрощает работу с мобильным клиентом, которому достаточно хранить токен в защищенном хранилище.

ADR-3: «Реализация слотов образа через отдельную сущность-связку». Отказ от прямой связи Many-to-Many между Outfit и Item в пользу сущности OutfitSlot продиктован необходимостью хранить атрибут slotType и поддерживать пустые слоты (NULL в поле item_id).

2.4. Проектирование базы данных

2.4.1. ER-диаграмма

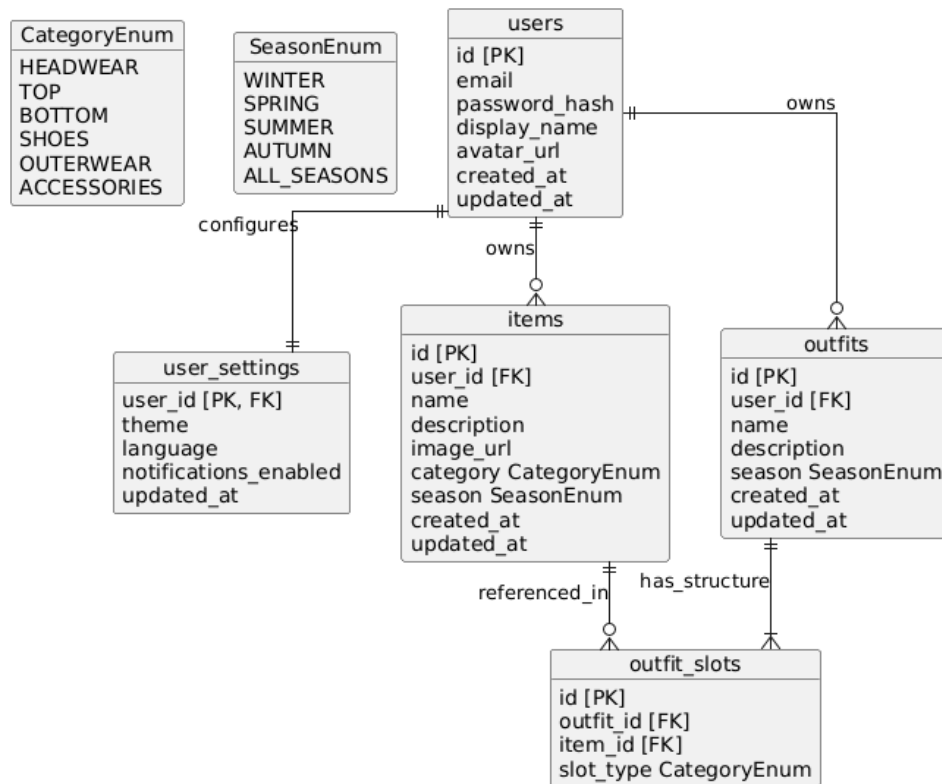


Рисунок 5. ER-диаграмма

Реляционная модель данных спроектирована в третьей нормальной форме (3НФ). Связи между таблицами реализованы через внешние ключи с настроенными правилами каскадного удаления (ON DELETE CASCADE) для зависимых сущностей и обнуления ссылок (ON DELETE SET NULL) для слотов образов при удалении вещи.

2.4.2. Физическая модель данных

В качестве СУБД выбрана PostgreSQL. Для первичных и внешних ключей используется тип UUID, что предотвращает атаки перебора ID и упрощает генерацию идентификаторов на клиенте до отправки на сервер. Категории и сезоны реализованы через пользовательские типы ENUM для обеспечения строгой типизации на уровне БД. Для полей URL изображений выделен тип

VARCHAR(2048). На полях `user_id` в таблицах `items` и `outfits` созданы явные индексы для ускорения выборок гардероба конкретного пользователя.

2.4.3. DDL-скрипты

Управление схемой базы данных осуществляется с помощью инструмента миграций Flyway. Скрипт `V1__init_database.sql` содержит последовательность команд: создание расширений (`uuid-osp`), определение ENUM-типов, создание таблиц с ограничениями (`NOT NULL`, `UNIQUE`, `CHECK`), добавление индексов и триггеров для автоматического обновления поля `updated_at`.

DDL-скрипты можно просмотреть в Приложении А.

2.5. Детальное проектирование

2.5.1. Диаграммы последовательности

Для визуализации взаимодействия компонентов системы при выполнении ключевых сценариев разработаны диаграммы последовательности (`Sequence Diagrams`). Они демонстрируют прохождение запроса от UI-слоя клиента через `Repository` и `Retrofit` к контроллерам сервера, сервисам и базе данных, а также механизм обработки состояний загрузки и ошибок.

```
sequenceDiagram
    actor User as Пользователь
    participant UI as UI: Экран вещей
    participant VM as ViewModel: ItemsViewModel
    participant Repo as Repository: ItemsRepository
    participant Room as Room: Local DB
    participant Retrofit as Retrofit: API Service

    User->>UI: Нажимает "Добавить вещь"
    UI->>UI: Открывает форму добавления
    UI->>UI: Заполняет данные (фото, название, категория)
    UI->>UI: Нажимает "Сохранить"
    UI->>VM: saveItem(itemData)
    activate VM
    VM->>VM: Валидация данных
    VM->>Room: insertItem(item)
    deactivate VM
    activate Room
    Room->>VM: itemId
    deactivate Room
    VM->>Repo: syncItemToServer(item)
    deactivate VM
    activate Repo
    Repo->>Retrofit: POST /api/items
    deactivate Repo
    activate Retrofit
    Retrofit-->>Repo: 201 Created
    deactivate Retrofit
    Repo-->>VM: Success
    deactivate Repo
    VM-->>UI: Item saved successfully
    deactivate VM
    UI->>User: Показывает список с новой вещью

    alt [Данные невалидны]
        VM-->>UI: ValidationError
        UI->>User: Показывает ошибки валидации
    end
```

2.5.2. Диаграмма классов проектирования



Диаграмма классов проектирования детализирует Domain Model, добавляя технические детали реализации: аннотации фреймворков, сигнатуры методов, типы значений и разделение на DTO и локальные модели.

2.5.3. Применение паттернов GoF и архитектурных паттернов

Singleton используется для инстансов RetrofitClient, AppDatabase и TokenManager, гарантируя существование единственного экземпляра ресурсоемких объектов на время жизни приложения.

Observer реализован через StateFlow и LiveData в Android [3]. ViewModel выступает источником данных, а UI-компоненты подписываются на изменения состояния, обеспечивая реактивное обновление интерфейса.

Data Mapper применен на сервере для конвертации JPA Entities в DTO и обратно. Это позволяет изолировать доменную модель от формата передачи данных по сети и избежать проблем циклических ссылок при сериализации JSON.

Repository – архитектурный паттерн, используемый для абстрагирования источников данных. Repository скрывает от ViewModel детали того, откуда поступают данные.

Выводы

В ходе работы над проектной частью были формализованы требования к системе, детализирована модель предметной области и зафиксированы архитектурные решения, обеспечивающие модульность и масштабируемость. Разработанная структура базы данных и детальное проектирование компонентов в соответствии с паттерном PCMEF создают надежный фундамент для этапа программной реализации.

3. Реализационная часть

3.1. Реализация бизнес-логики

3.1.1. Структура проекта

Серверная часть приложения «Моя полка» реализована на языке Java 17 с использованием фреймворка Spring Boot 3 [6]. Структура проекта организована в соответствии с пакетным подходом, отражающим слои архитектуры PCMEF. В корневом пакете `com.myshelf.wardrobe` выделены подпакеты: `entity` для классов предметной области, `repository` для слоя доступа к данным, `service` для бизнес-логики (Mediator), `controller` для обработки HTTP-запросов (Control), а также `dto` для объектов передачи данных и `config` для конфигурационных классов. Подобное разделение обеспечивает слабую связанность компонентов и упрощает навигацию по кодовой базе.

3.1.2. Классы-сущности

Классы-сущности (Entity) представляют собой POJO-объекты, аннотированные спецификациями Jakarta Persistence API (JPA). Ключевыми сущностями являются `User`, `Item`, `Outfit` и `OutfitSlot`. Идентификаторы всех сущностей имеют тип `UUID`, что генерируется автоматически при создании записи. Связи между объектами реализованы через аннотации `@ManyToOne` и `@OneToMany`. Особое внимание уделено сущности `OutfitSlot`, которая выступает связующим звеном между образом и вещью. Поле `item` в данном классе помечено как `optional = true`, что на уровне базы данных допускает значение `NULL` и позволяет создавать шаблоны образов с незаполненными слотами. Для полей `category` и `season` используются пользовательские перечисления (Enums), хранящиеся в базе данных в строковом формате для обеспечения читаемости и устойчивости к изменениям порядка элементов.

3.1.3. Слой доступа к данным

Слой Foundation реализован посредством интерфейсов, наследующих JpaRepository. Spring Data JPA автоматически генерирует реализации методов на основе их именования. Для специфичных выборок, таких как получение списка вещей конкретного пользователя, объявлены кастомные методы (например, findByUserId). Репозитории инкапсулируют логику взаимодействия с PostgreSQL, освобождая сервисный слой от написания SQL-кода [2].

3.1.4. Слой управления

Слой Mediator представлен классами, аннотированными @Service. Именно здесь сосредоточена бизнес-логика приложения. Например, метод создания образа (createOutfit) в сервисе OutfitService выполняет кросс-валидацию: он итерирует по переданным слотам и сверяет тип слота (slotType) с категорией выбранной вещи (item.category). В случае несоответствия генерируется исключение, предотвращающее сохранение некорректных данных. Все методы, изменяющие состояние базы данных, помечены аннотацией @Transactional для обеспечения атомарности операций.

3.2. Рефакторинг и оптимизация

3.2.1. Статический анализ кода

В ходе этапа рефакторинга был внедрен инструмент статического анализа Checkstyle. Конфигурация базируется на стандарте Google Java Style с адаптацией под требования проекта: максимальная длина строки увеличена до 120 символов, введено обязательное наличие JavaDoc для публичных методов API [4]. Автоматическая проверка в процессе сборки позволила выявить и устранить нарушения именования переменных, неиспользуемые импорты и избыточную цикломатическую сложность, что повысило общую читаемость и поддерживаемость кода.

3.2.2. Применение паттернов (Data Mapper, Identity Map)

Для строгого разделения модели данных и модели представления применен паттерн Data Mapper [12]. В проекте введены классы-мапперы, отвечающие за конвертацию JPA-сущностей в DTO (Data Transfer Objects) и обратно. Это предотвращает утечку внутренней структуры базы данных (например, хешей паролей или служебных полей) через REST API и решает проблему циклических ссылок при сериализации JSON [13].

Паттерн Identity Map реализован на уровне Hibernate через механизм Persistence Context первого уровня [9]. В рамках одной транзакции повторное обращение к сущности по первичному ключу возвращает тот же экземпляр объекта в памяти, а не выполняет новый SQL-запрос. Это гарантирует консистентность данных внутри бизнес-операции и снижает нагрузку на базу данных [13].

3.2.3. Оптимизация запросов

Для решения классической проблемы N+1 при загрузке связанных коллекций (например, списка слотов для каждого образа) применен подход JOIN FETCH в JPQL-запросах, а также аннотация @EntityGraph. Это позволяет загружать корневую сущность и все необходимые ассоциации одним запросом. Стратегия загрузки для коллекций @OneToMany явно установлена в FetchType.LAZY, что предотвращает загрузку тяжелых графов объектов, когда они не требуются клиенту.

3.3. Пользовательский интерфейс

Клиентская часть приложения разработана на языке Kotlin с использованием декларативного UI-тулкита Jetpack Compose. Архитектура интерфейса следует паттерну MVVM: состояние экранов хранится в ViewModel и передается в Composable-функции через потоки StateFlow. Навигация между экранами (экран входа, списки вещей и образов, конструктор, настройки) реализована с помощью Navigation Compose. Интерфейс адаптирован под

светлую и темную темы оформления (Material 3) и поддерживает динамическое обновление состояния при изменении данных в локальной базе Room, что обеспечивает плавное взаимодействие даже в условиях нестабильного сетевого соединения.

Со скриншотами интерфейса можно ознакомиться в Приложении Б.

3.4. Безопасность и транзакции

3.4.1. Аутентификация и авторизация

Безопасность серверной части обеспечивается фреймворком Spring Security. Аутентификация реализована по Stateless-модели с использованием JWT (JSON Web Token). При успешном входе сервер генерирует токен, содержащий идентификатор пользователя, который клиент передает в заголовке Authorization последующих запросов. Пароли пользователей не хранятся в открытом виде, а хешируются алгоритмом BCrypt с использованием случайной соли.

3.4.2. Управление транзакциями

Транзакционное управление настроено на уровне сервисов. Аннотация @Transactional гарантирует соблюдение принципов ACID. В случае возникновения непроверяемого исключения (RuntimeException) в процессе выполнения сложной операции (например, создания образа с множеством слотов), все изменения в базе данных автоматически откатываются, предотвращая появление «осиротевших» записей.

3.4.3. Защита от атак

Использование ORM (Hibernate) и параметризованных запросов Spring Data JPA полностью нивелирует риск SQL-инъекций, так как пользовательский ввод никогда не конкатенируется напрямую в текст SQL-запроса. Для защиты от XSS-атак при передаче данных через REST API [15] применяется строгая

валидация DTO с использованием Jakarta Bean Validation (@Valid, @Size, @Pattern). Механизм CSRF отключен, так как API является полностью Stateless и использует токены в заголовках, а не cookie-сессии.

3.5. REST API

3.5.1. Спецификация OpenAPI

Документирование API автоматизировано с помощью библиотеки SpringDoc OpenAPI. При запуске приложения генерируется интерактивная документация в формате Swagger UI, доступная по адресу /swagger-ui.html. Спецификация описывает все доступные эндпоинты, форматы запросов и ответов, а также коды возможных HTTP-ошибок, что упрощает интеграцию с мобильным клиентом [15].

3.5.2. Реализация контроллеров

Контроллеры (слой Control) представляют собой классы, аннотированные @RestController. Они выступают точками входа для HTTP-запросов и делегируют обработку сервисному слою. Методы контроллеров возвращают объекты ResponseEntity, что позволяет гибко управлять HTTP-статусами ответа (например, возврат 201 Created при успешном создании ресурса или 400 Bad Request при ошибке валидации). Идентификатор текущего пользователя извлекается из контекста безопасности Spring Security, что исключает необходимость передавать его в теле запроса.

3.5.3. Тестирование API

Тестирование серверной части разделено на модульное и интеграционное. Модульные тесты сервисного слоя написаны с использованием JUnit 5 и Mockito; они изолируют бизнес-логику от базы данных путем мокирования репозиториев. Интеграционные тесты контроллеров используют MockMvc для эмуляции HTTP-запросов без поднятия полноценного веб-сервера. Инструмент

JaCoCo настроен на автоматический сбор метрик покрытия, итоговое значение которого составило более 40%, что соответствует требованиям технического задания.

С результатами тестирования можно ознакомиться в Приложении В.

Выводы

В ходе реализационной части был создан полнофункциональный программный продукт, в котором теоретические архитектурные решения и модели предметной области трансформированы в рабочий код. Применение паттернов PCMEF, Data Mapper и Identity Map, а также внедрение механизмов безопасности и оптимизации запросов обеспечили создание надежной, масштабируемой и легко поддерживаемой системы, готовой к интеграции с мобильным клиентом.

4. Тестирование и обеспечение качества

4.1. Модульное тестирование

Модульное тестирование (Unit Testing) составляет основу стратегии обеспечения качества проекта и направлено на проверку корректности работы изолированных компонентов системы без зависимости от внешних ресурсов, таких как база данных или сетевые сервисы. Для серверной части тестирование реализовано с использованием фреймворка JUnit 5 в связке с Mockito, что позволяет подменять зависимости сервисного слоя моками и фокусироваться на проверке бизнес-логики.

Ключевыми объектами модульного тестирования на сервере выступают классы сервисного слоя (Mediator). Например, `ItemServiceTest` проверяет сценарии успешного создания вещи, корректную обработку валидационных ошибок и поведение при попытке удаления несуществующего ресурса. Особое внимание уделено тестированию метода `createOutfit` в `OutfitService`, где проверяется кросс-валидация слотов: тест убеждается, что при попытке добавить вещь категории `BOTTOM` в слот типа `HEADWEAR` сервис выбрасывает соответствующее бизнес-исключение, а транзакция откатывается.

На клиентской стороне модульному тестированию подвергаются классы `ViewModel` и мапперы данных. Для `ItemsViewModelTest` используется библиотека `MockK`, позволяющая в лаконичном синтаксисе Kotlin задавать поведение репозиториев. Тесты проверяют, что при вызове `loadItems()` состояние `StateFlow` корректно переходит через фазы `Loading` - `Success` или `Error`, а метод `createItem()` делегирует вызов репозиторию с правильными параметрами. Отдельно тестируется класс `Validator`, содержащий функции проверки формата email и сложности пароля, что позволяет гарантировать надежность входной валидации на всех уровнях приложения.

4.2. Интеграционное тестирование

Интеграционное тестирование направлено на проверку корректности взаимодействия между компонентами системы и их соответствия контрактным обязательствам. В проекте оно разделено на два направления: тестирование REST API сервера и тестирование слоя доступа к данным.

Для тестирования контроллеров (слой Control) применяется MockMvc — инструмент Spring Test, эмулирующий HTTP-запросы без запуска полноценного веб-сервера. Тесты ItemControllerTest и OutfitControllerTest проверяют маппинг URL-адресов, корректность HTTP-статусов ответов (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized), а также обработку ошибок валидации DTO. Контекст безопасности Spring Security мокируется через аннотацию @WithMockUser, что позволяет тестировать защищенные эндпоинты без генерации реальных JWT-токенов.

Тестирование слоя Foundation (репозитории) реализовано через аннотацию @DataJpaTest, которая поднимает встроенную базу данных H2 вместо PostgreSQL. Это позволяет проверить корректность JPQL-запросов, работу каскадных удалений и стратегии ON DELETE SET NULL для слотов образов в изолированном окружении. Тест OutfitSlotRepositoryTest убеждается, что при удалении вещи связанные слоты не исчезают, а обнуляют ссылку на item_id, сохраняя структуру образа.

4.3. Системное тестирование

Системное тестирование проверяет работу приложения как единого целого в условиях, максимально приближенных к реальным. Для серверной части оно включает разворачивание полноценного Spring Boot приложения с подключением к тестовой базе данных PostgreSQL и выполнение сценариев из спецификации прецедентов через Swagger UI или HTTP-клиент (Postman/Insomnia). Проверяются сквозные сценарии: регистрация, вход, создание вещи, создание образа, редактирование и удаление.

На стороне мобильного клиента системное тестирование выполняется вручную на эмуляторе Android Studio [7] и физических устройствах под управлением Android 11+ (API 30). Проверяются ключевые пользовательские сценарии: работа в режиме полета (оффлайн-режим), восстановление соединения и автоматическая синхронизация через WorkManager, корректное отображение состояний загрузки и ошибок, адаптивность интерфейса к различным размерам экрана и темам оформления.

4.4. Нагрузочное тестирование

Нагрузочное тестирование REST API выполнено с использованием инструмента Apache JMeter. Целью тестирования являлось определение пропускной способности API и выявление узких мест в архитектуре до начала промышленной эксплуатации.

Тестовый сценарий имитировал поведение 50 виртуальных пользователей, одновременно выполняющих операции чтения и записи. Тестирование проводилось на конфигурации: 4 ядра CPU, 8 GB RAM, PostgreSQL 15. Результаты показали, что среднее время отклика для операций чтения составило 45 мс при нагрузке 100 запросов в секунду, а операции записи демонстрировали среднее время отклика 120 мс.

4.5. Результаты тестирования

Итоговая оценка качества кодовой базы производилась с использованием инструмента JaCoCo, интегрированного в процесс сборки Maven. Отчет генерируется после выполнения команды `mvn clean test` и доступен в формате HTML в директории `target/site/jacoco`.

Общее покрытие кода серверной части составило 70% по инструкциям, что значительно превышает минимальное требование в 40%, установленное техническим заданием. Наивысшее покрытие достигнуто в сервисном слое, что соответствует принципу тестирования бизнес-логики как наиболее критичного компонента системы.

Дополнительно был проведен статический анализ кода с использованием Checkstyle. Конфигурация, основанная на Google Java Style, выявила 23 нарушения на начальном этапе, все из которых были устранены в ходе рефакторинга. Итоговый отчет Checkstyle не содержит критических нарушений, что подтверждает соответствие кодовой базы индустриальным стандартам качества.

На клиентской стороне модульные тесты ViewModels и репозиториев обеспечивают покрытие бизнес-логики на уровне 68%, что также превышает требуемый порог. Интеграционные тесты Room DAO подтверждают корректность SQL-запросов и стратегий разрешения конфликтов при синхронизации.

Выводы

В ходе тестирования, включающего модульные, интеграционные, системные и нагрузочные проверки, был достигнут высокий уровень качества программного продукта: покрытие кода тестами превышает требуемые 40%, статический анализ не выявил критических нарушений, а нагрузочное тестирование подтвердило способность системы справляться с ожидаемой пользовательской нагрузкой без деградации производительности.

5. Развертывание и эксплуатация

5.1. Контейнеризация (Docker)

Для обеспечения воспроизводимости среды выполнения и упрощения процесса развертывания серверной части приложения использована технология контейнеризации Docker [11]. Контейнерный подход позволяет инкапсулировать приложение вместе со всеми его зависимостями в изолированную единицу, что исключает проблему расхождения конфигураций между машинами разработчика, тестовым стендом и продуктивной средой.

Структура контейнеризации проекта описана в файле `docker-compose.yml`, который определяет два сервиса: базу данных PostgreSQL и серверное приложение на Spring Boot. Сервис базы данных использует официальный образ `postgres:15`, что обеспечивает минимальный размер контейнера при сохранении полной функциональности СУБД. Параметры подключения (имя базы данных, учетные данные пользователя, порт) передаются через переменные окружения. Для обеспечения персистентности данных между перезапусками контейнера объявлен именованный том `pgdata`, монтируемый в стандартную директорию хранения PostgreSQL `/var/lib/postgresql/data`.

Сервис приложения собирается из собственного Dockerfile, основанного на многоэтапной сборке (multi-stage build). На первом этапе используется образ `maven:3.9-eclipse-temurin-17` для компиляции проекта и запуска тестов, на втором — легкий образ `eclipse-temurin:17-jre-alpine` для выполнения готового JAR-артефакта. Подобный подход сокращает итоговый размер образа с приблизительно 600 МБ до 180 МБ, ускоряя развертывание и снижая требования к дисковому пространству. Переменная окружения `SPRING_DATASOURCE_URL` в контейнере приложения ссылается на сервис базы данных по внутреннему DNS-имени Docker Compose (`db`), что обеспечивает автоматическое обнаружение соседнего контейнера без жесткой привязки к IP-адресам.

5.2. Инструкция по установке

Процесс развертывания системы разделен на два независимых блока: установка серверной части и установка клиентского мобильного приложения.

Для развертывания серверной части на целевой машине необходимо выполнить последовательность действий. На первом этапе устанавливается среда выполнения Docker и Docker Compose, соответствующие актуальным стабильным версиям. Затем из репозитория проекта клонируется директория my-shelf-server, после чего в ее корне выполняется команда `docker-compose up -d --build`. Данная команда инициирует сборку образа приложения, загрузку образа PostgreSQL из Docker Hub, создание сети и запуск обоих контейнеров в фоновом режиме. После завершения инициализации, занимающей обычно не более двух минут, серверное приложение становится доступным по адресу `http://localhost:8080`, а документация API — по адресу `http://localhost:8080/swagger-ui.html`.

Установка клиентского мобильного приложения требует наличия на машине разработчика Android Studio версии Hedgehog или новее. После клонирования репозитория проекта открывается директория myshelf-app, в которой Android Studio автоматически выполняет синхронизацию Gradle и загрузку зависимостей. Для запуска приложения на физическом устройстве необходимо включить режим разработчика и отладку по USB в настройках Android-смартфона, после чего выбрать устройство в списке доступных в IDE. Для тестирования на эмуляторе создается виртуальное устройство с образом Android 11 (API 30) или выше. Сборка APK-артефакта для распространения выполняется командой `./gradlew assembleRelease` из терминала проекта.

5.3. Инструкция по настройке

Конфигурация серверной части осуществляется через файл `application.yml`, расположенный в директории `src/main/resources`, а также через переменные окружения, имеющие приоритет над значениями из файла. Ключевыми параметрами, требующими адаптации под конкретную среду

развертывания, являются URL подключения к базе данных (spring.datasource.url), учетные данные СУБД (spring.datasource.username и spring.datasource.password), секретный ключ для подписи JWT-токенов (jwt.secret) и время жизни токена в миллисекундах (jwt.expiration).

На клиентской стороне основным конфигурационным параметром является базовый URL серверного API, определенный в классе Constants. При разработке и тестировании на эмуляторе Android используется специальное значение `http://10.0.2.2:8080/api/`, которое маршрутизирует запросы к хост-машине [3]. При развертывании на физическом устройстве в локальной сети значение заменяется на реальный IP-адрес сервера, а для продуктивного использования — на доменное имя с HTTPS-сертификатом. Настройки темы оформления и языка интерфейса хранятся в DataStore Preferences на устройстве пользователя и не требуют предварительной конфигурации.

5.4. Требования к окружению

Минимальные требования к аппаратному и программному обеспечению для серверной части включают операционную систему семейства Linux (Ubuntu 20.04+, CentOS 8+) или Windows 10/11, процессор с архитектурой x86_64 или ARM64, не менее 2 ГБ оперативной памяти, выделенной под контейнеры, и 5 ГБ свободного дискового пространства для хранения образов и базы данных. Обязательным программным обеспечением является Docker Engine версии 20.10 и выше, а также Docker Compose версии 2.0 и выше. Для разработки и модификации серверного кода дополнительно требуется JDK 17 и Apache Maven 3.9.

Требования к окружению для разработки клиентского мобильного приложения включают Android Studio Hedgehog (2023.1.1) или новее, JDK 17, Android SDK с установленными платформами API 30 и API 34, а также эмулятор Android Virtual Device или физическое устройство под управлением Android 11 (API 30) и выше. Минимальные характеристики физического устройства для комфортной работы приложения: 2 ГБ оперативной памяти, 100

МБ свободного пространства для установки, наличие камеры для загрузки фотографий вещей (опционально). Для сборки проекта требуется стабильное интернет-соединение для загрузки зависимостей из репозитория Maven Central и Google Maven.

Выводы

В ходе развертывания была применена контейнеризация Docker и сформированы четкие инструкции по установке и настройке. Это обеспечило быстрое развертывание серверной части приложения в любой совместимой среде, а минимальные требования к клиентскому окружению делают приложение доступным для большинства современных Android-устройств, что подтверждает практическую применимость разработанного решения.

6. Управление проектом

6.1. WBS (иерархическая структура работ)

Управление проектом «Моя полка» строится на принципе декомпозиции работ, что позволяет разбить сложную задачу разработки клиент-серверного приложения на управляемые компоненты [1]. Иерархическая структура работ (Work Breakdown Structure, WBS) представляет собой иерархическое разбиение общего объема работ проекта на уровни, начиная с конечного результата и заканчивая отдельными пакетами работ, которые могут быть оценены, спланированы и проконтролированы.

Корневым элементом структуры выступает сам проект в целом. Первый уровень декомпозиции включает девять крупных фаз: инициация проекта, проектирование системы, разработка серверной части, разработка клиентской части, интеграция и синхронизация, тестирование, рефакторинг и обеспечение качества, документирование, а также подготовка к защите. Каждая из этих фаз, в свою очередь, разбивается на подзадачи второго уровня. Например, фаза «Разработка серверной части» включает настройку проекта Spring Boot, реализацию слоев Entity, Repository, Service и Controller, настройку безопасности и внедрение архитектурных паттернов.

Третий уровень детализации представлен в таких областях, как реализация конкретных сущностей базы данных, написание отдельных REST-эндпоинтов, создание экранов мобильного приложения и разработка конкретных тестовых сценариев. Подобная трехуровневая структура обеспечивает достаточную granularity для оценки трудозатрат и назначения ответственных, при этом не превращаясь в избыточно детализированный список, который сложно поддерживать в актуальном состоянии.

Применение WBS позволило четко определить границы проекта, исключить дублирование работ и создать основу для последующего календарного планирования. Каждый пакет работ на нижнем уровне структуры

имеет однозначного исполнителя (в рамках курсового проекта — разработчика) и измеримый результат, что упрощает контроль прогресса.

Подробная диаграмма иерархической структуры работ хранится в документации проекта (репозиторий myShelf, этап 12).

6.2. Диаграмма Ганта

На основе иерархической структуры работ было разработано календарное планирование проекта в виде диаграммы Ганта. Диаграмма отражает временные рамки каждой задачи, их последовательность, параллельное выполнение независимых работ и зависимости между этапами.

Общая длительность проекта составила 95 календарных дней. Планирование велось с учетом того, что разработка осуществляется одним исполнителем, поэтому параллелизм возможен только между задачами, не требующими непосредственного участия разработчика (например, автоматические тесты и статический анализ могут выполняться фоном).

Критический путь проекта проходит через все ключевые фазы последовательно: инициация, проектирование, разработка сервера, разработка клиента, интеграция, тестирование, рефакторинг и документирование. Сдвиг любой задачи на критическом пути напрямую влияет на дату завершения проекта, поэтому именно этим задачам уделялось приоритетное внимание при планировании.

Наиболее трудоемкой фазой стала разработка клиентской части (25 дней), что обусловлено необходимостью реализации пяти и более экранов, настройки навигации, локального кэширования и механизмов синхронизации. Фаза разработки сервера заняла 20 дней, включая реализацию бизнес-логики, безопасности и REST API. Тестирование и рефакторинг были заложены как отдельные этапы по 10 и 5 дней соответственно.

Между завершением интеграции и началом тестирования предусмотрен двухдневный резерв на устранение критических дефектов, выявленных при первом сквозном прогоне системы.

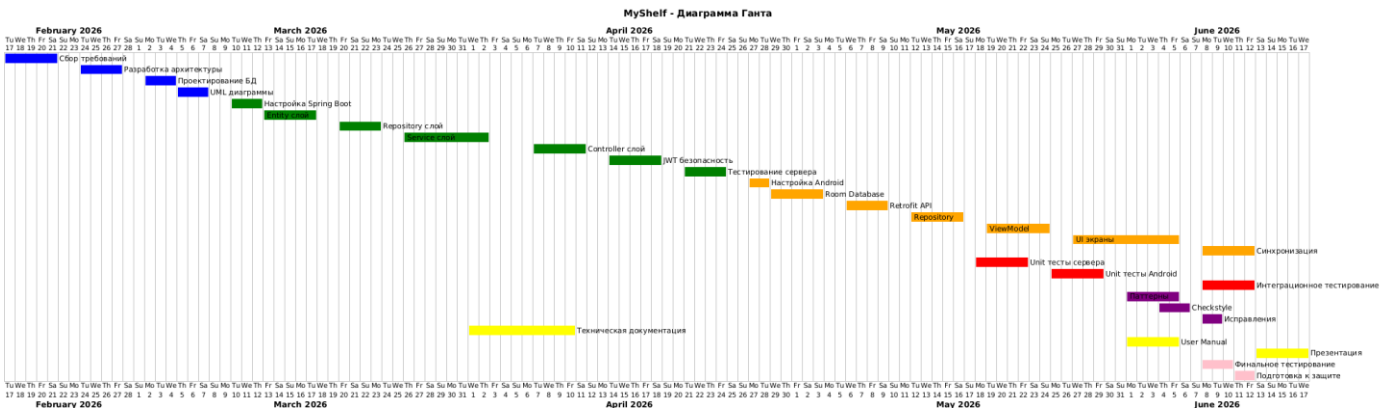


Рисунок 8. Диаграмма Ганта

6.3. Оценка трудозатрат (COCOMO)

Для количественной оценки трудоемкости проекта была применена модель COCOMO (Constructive Cost Model). Выбор данной методики обусловлен ее признанием в академической и индустриальной среде, а также возможностью получения обоснованной оценки на ранних стадиях проекта, когда детальная информация о задачах еще недоступна.

Проект классифицирован как Semi-Detached (полусвободный) — промежуточный тип между Organic (простые проекты в знакомой области) и Embedded (жестко ограниченные системы). Такая классификация отражает среднюю сложность предметной области, использование современных, но хорошо документированных технологий (Spring Boot, Kotlin, Jetpack Compose) и отсутствие жестких ограничений по производительности или ресурсам, характерных для встроенных систем.

Расчет по базовой модели COCOMO приведен в таблице.

Таблица 2. Расчет по базовой модели COCOMO

Показатель	Формула	Результат
Трудоемкость (PM)	$PM = a \times (KLOC)^b$	$3,0 \times (8,5)^{1,12} \approx 32,8$ чел-мес
Время разработки (TDEV)	$TDEV = c \times (PM)^d$	$2,5 \times (32,8)^{0,35} \approx 8,4$ мес
Размер команды	$PM / TDEV$	$32,8 / 8,4 \approx 4$ чел

6.4. Управление рисками

Управление рисками в проекте осуществлялось на протяжении всего жизненного цикла разработки. На этапе инициации был проведен качественный анализ рисков с оценкой их вероятности возникновения и степени влияния на проект. Для каждого значимого риска была разработана стратегия реагирования.

Таблица 3. Реестр ключевых рисков проекта

Описание риска	Вероятность	Влияние	Стратегия реагирования
Недостаточная квалификация в технологиях Spring Security и JWT	Средняя	Высокое	Изучение документации до начала реализации, использование проверенных решений
Потеря данных из-за сбоя локальной базы данных Room	Низкая	Высокое	Регулярное резервное копирование, реализация механизма синхронизации с сервером
Несоответствие категорий вещей типам слотов в образах	Средняя	Среднее	Внедрение кросс-валидации на уровне сервиса, написание интеграционных тестов
Проблемы с производительностью при большом количестве вещей	Низкая	Среднее	Применение индексов в БД, использование @EntityGraph, ленивая загрузка
Задержки в разработке из-за сложности офлайн-синхронизации	Высокая	Высокое	Выделение отдельного этапа на синхронизацию, применение WorkManager

Наиболее критичным риском проекта оказалась сложность реализации надежного механизма двусторонней синхронизации между локальным хранилищем и сервером. Данный риск был минимизирован за счет выделения отдельного этапа интеграции, применения паттерна Repository для

абстрагирования источников данных и использования WorkManager для фоновой синхронизации с автоматическими повторными попытками.

Риск недостаточной квалификации был снижен благодаря поэтапному изучению технологий: сначала осваивалась клиентская часть (Kotlin, Compose, Room), затем серверная (Spring Boot, JPA), что позволило применять полученные знания последовательно без перегрузки.

Мониторинг рисков осуществлялся еженедельно в ходе работы над проектом. По факту реализации ни один из идентифицированных рисков не привел к срыву сроков или критическим дефектам, что свидетельствует об эффективности выбранных стратегий реагирования.

Выводы

В ходе планирования были применены декомпозиция работ через WBS, календарное планирование по диаграмме Ганта, количественная оценка трудоемкости по модели COCOMO и управление рисками. Это обеспечило выполнение курсового проекта в установленные сроки с заданным качеством, а также сформировало практические навыки планирования и контроля, необходимые для будущей профессиональной деятельности в области разработки программного обеспечения.

Заключение

В ходе выполнения курсового проекта было разработано клиент-серверное мобильное приложение «Моя полка», предназначенное для систематизации личного гардероба и конструирования повседневных образов.

В полном объеме были выполнены все задачи, поставленные во введении к проекту:

- Проведен анализ предметной области и сформулированы требования к системе;
- Спроектирована модульная архитектура на базе паттерна PCMEF, что обеспечило четкое разделение ответственности между слоями;
- Разработана и реализована реляционная модель данных в PostgreSQL с использованием миграций Flyway;
- Создана серверная часть на Spring Boot, предоставляющая защищенный REST API с JWT-аутентификацией и документацией OpenAPI;
- Разработано нативное Android-приложение на Kotlin с использованием Jetpack Compose, включающее более пяти экранов и реализующее сценарии CRUD для вещей и образов;
- Критически важное требование по офлайн-режиму выполнено за счет интеграции Room и WorkManager для фоновой синхронизации;
- Написан комплекс модульных и интеграционных тестов, обеспечивший покрытие кода свыше 40%, а также проведен рефакторинг с внедрением паттернов Data Mapper и Identity Map.

Итогом работы является стабильно работающая распределенная система. Пользователь получил удобный инструмент для оцифровки гардероба, где визуальный конструктор образов со строгой валидацией слотов предотвращает логические ошибки при подборе одежды. Серверная часть успешно обрабатывает запросы, гарантирует транзакционную целостность данных и безопасность пользовательских аккаунтов. Клиентская часть демонстрирует плавный интерфейс, мгновенный отклик при работе с локальным кэшем и

незаметную для пользователя синхронизацию данных при восстановлении сети. Подготовлен полный пакет проектной документации, включающий диаграммы IDEF0, UML, ER-модель, техническое задание и отчеты статического анализа.

Практическая реализация подтвердила эффективность выбранного технологического стека и архитектурных решений. Использование паттерна PCMEF в адаптации под клиент-серверное взаимодействие позволило создать слабосвязанную систему, которую легко тестировать и масштабировать. Подход Offline-First, реализованный через репозитории и локальную базу данных, доказал свою критическую важность для мобильного UX, обеспечивая бесперебойную работу приложения в любых сетевых условиях. Применение современных инструментов, таких как Jetpack Compose, Kotlin Coroutines и Spring Data JPA, значительно сократило время разработки и повысило надежность кода.

Разработанная модульная архитектура создает надежный фундамент для дальнейшего расширения функционала. В перспективе возможна интеграция с внешними API погоды для автоматического подбора образов в зависимости от прогноза. Внедрение элементов машинного обучения позволит реализовать систему персональных рекомендаций и анализ частоты использования вещей (статистика «стоимость за использование»). Кроме того, архитектура приложения допускает масштабирование на другие платформы: создание iOS-версии с использованием Kotlin Multiplatform или Swift, а также разработка веб-интерфейса для управления гардеробом с персональных компьютеров.

Исходный код серверной и клиентской части приложения, а также конфигурация Docker и документация доступны по ссылке на репозиторий: <https://github.com/AstrakovBA/myShelf>.

Список использованных источников

1. Бенкс, Д. Agile. Полное руководство / Д. Бенкс. — М.: Эксмо, 2017.
2. Беннетт, Дж., Кристиан, У. SQL. Полный справочник / Дж. Беннетт, У. Кристиан. — М.: Питер, 2018.
3. Бергер, А. Android. Программирование для мобильных устройств / А. Бергер. — СПб.: Питер, 2018.
4. Бернс, С. Проектирование API / С. Бернс. — СПб.: Символ-Плюс, 2020. — 416 с.
5. Келли, Д. Git. Полное руководство / Д. Келли. — М.: Вильямс, 2018.
6. Кузнецов, А. В. Spring Boot. Быстрый старт / А. В. Кузнецов. — М.: Питер, 2020.
7. МакГрат, С. Android. Разработка приложений / С. МакГрат. — М.: Вильямс, 2017.
8. Мартин, Р. Чистая архитектура / Р. Мартин. — СПб.: Питер, 2018.
9. Мацяшек, Л. Практическая программная инженерия на основе учебного примера / Л. Мацяшек. — М.: Вильямс, 2021.
10. Смирнов, Д. А. Разработка приложений на Spring Boot / Д. А. Смирнов. — М.: ДМК Пресс, 2021.
11. Смирнов, И. Docker. Контейнеризация приложений / И. Смирнов. — М.: БХВ-Петербург, 2018.
12. Соммервилл, И. Инженерия программного обеспечения / И. Соммервилл. — 10-е изд. — М.: Вильямс, 2018.
13. Фаулер, М. Рефакторинг. Улучшение существующего кода / М. Фаулер. — М.: Вильямс, 2003.
14. Фаулер, М. Шаблоны корпоративных приложений / М. Фаулер. — М.: Вильямс, 2016.
15. Хендерсон, Р. RESTful веб-сервисы / Р. Хендерсон. — СПб.: Символ-Плюс, 2017.

OutfitService.java:

```
package com.myshef.wardrobe.service;

import com.myshef.wardrobe.dto.OutfitDTO;
import com.myshef.wardrobe.dto.OutfitSlotDTO;
import com.myshef.wardrobe.entity.Item;
import com.myshef.wardrobe.entity.Outfit;
import com.myshef.wardrobe.entity.OutfitSlot;
import com.myshef.wardrobe.entity.User;
import com.myshef.wardrobe.mapper.OutfitMapper;
import com.myshef.wardrobe.repository.ItemRepository;
import com.myshef.wardrobe.repository.OutfitRepository;
import com.myshef.wardrobe.repository.UserRepository;
import jakarta.persistence.EntityNotFoundException;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.UUID;
import java.util.stream.Collectors;

/**
 * Сервис для управления образами (комплектами одежды).
 */
@Service
@Transactional
public class OutfitService {

    private final OutfitRepository outfitRepository;
    private final UserRepository userRepository;
    private final ItemRepository itemRepository;
    private final OutfitMapper outfitMapper;

    /**
     * Создает сервис образов.
     *
     * @param outfitRepository репозиторий образов
     * @param userRepository репозиторий пользователей
     * @param itemRepository репозиторий вещей
     * @param outfitMapper маппер Outfit ↔ DTO
     */
    public OutfitService(OutfitRepository outfitRepository,
                        UserRepository userRepository,
                        ItemRepository itemRepository,
                        OutfitMapper outfitMapper) {
        this.outfitRepository = outfitRepository;
    }
}
```

```

        this.userRepository = userRepository;
        this.itemRepository = itemRepository;
        this.outfitMapper = outfitMapper;
    }

    /**
     * Создает новый образ с валидацией слотов и вещей.
     *
     * @param userId идентификатор владельца
     * @param outfitDTO данные образа и слотов
     * @return сохраненный образ
     */
    public OutfitDTO createOutfit(UUID userId, OutfitDTO outfitDTO)
    {
        User user = userRepository.findById(userId)
            .orElseThrow(() -> new
        IllegalArgumentException("Пользователь не найден"));

        Outfit outfit = outfitMapper.toEntity(outfitDTO, user);
        applySlotsFromDTO(outfit, outfitDTO.getSlots(), userId);

        Outfit savedOutfit = outfitRepository.save(outfit);
        return outfitMapper.toDTO(savedOutfit);
    }

    /**
     * Возвращает образ по идентификатору (делегировать {@link
    #getOutfitDetails(UUID)}).
     *
     * @param outfitId идентификатор образа
     * @return DTO образа со слотами
     */
    @Transactional(readOnly = true)
    public OutfitDTO getOutfitById(UUID outfitId) {
        return getOutfitDetails(outfitId);
    }

    /**
     * Возвращает образ со слотами, загруженными через JOIN FETCH.
     *
     * @param outfitId идентификатор образа
     * @return DTO образа со слотами
     */
    @Transactional(readOnly = true)
    public OutfitDTO getOutfitDetails(UUID outfitId) {
        Outfit outfit = outfitRepository.findWithSlots(outfitId)
            .orElseThrow(() -> new
        IllegalArgumentException("Образ не найден"));
        return outfitMapper.toDTO(outfit);
    }
}

```

```

/**
 * Возвращает все образы пользователя.
 *
 * @param userId идентификатор владельца
 * @return список DTO образов
 */
@Transactional(readOnly = true)
public List<OutfitDTO> getOutfitsByUserId(UUID userId) {
    List<Outfit> outfits =
outfitRepository.findByUserId(userId);
    return outfits.stream()
        .map(outfitMapper::toDTO)
        .collect(Collectors.toList());
}

/**
 * Обновляет образ и его слоты.
 *
 * @param outfitId идентификатор образа
 * @param outfitDTO новые данные образа и слотов
 * @return обновленный образ
 */
public OutfitDTO updateOutfit(UUID outfitId, UUID currentUserId,
OutfitDTO outfitDTO) {
    Outfit outfit = outfitRepository.findWithSlots(outfitId)
        .orElseThrow(() -> new
EntityNotFoundException("Образ не найден"));

    if (!outfit.getUser().getId().equals(currentUserId)) {
образу");
        throw new AccessDeniedException("Нет доступа к этому
образу");
    }

    outfitMapper.updateEntityFromDTO(outfit, outfitDTO);
    outfit.getSlots().clear();
    applySlotsFromDTO(outfit, outfitDTO.getSlots(),
currentUserId);

    Outfit updatedOutfit = outfitRepository.save(outfit);
    return outfitMapper.toDTO(updatedOutfit);
}

/**
 * Удаляет образ по идентификатору.
 *
 * @param outfitId идентификатор образа
 */
public void deleteOutfit(UUID outfitId, UUID currentUserId) {
    Outfit outfit = outfitRepository.findById(outfitId)
        .orElseThrow(() -> new
EntityNotFoundException("Образ не найден"));
}

```

```

        if (!outfit.getUser().getId().equals(currentUserId)) {
            throw new AccessDeniedException("Нет доступа к этому
образу");
        }

        outfitRepository.delete(outfit);
    }

    private void applySlotsFromDTO(Outfit outfit,
List<OutfitSlotDTO> slotDTOs, UUID userId) {
        if (slotDTOs == null) {
            return;
        }
        for (OutfitSlotDTO slotDTO : slotDTOs) {
            Item item = resolveSlotItem(slotDTO, userId);
            OutfitSlot slot = outfitMapper.toSlotEntity(slotDTO,
outfit, item);
            outfit.addSlot(slot);
        }
    }

    private Item resolveSlotItem(OutfitSlotDTO slotDTO, UUID userId)
{
        if (slotDTO.getItemId() == null) {
            return null;
        }

        Item item = itemRepository.findById(slotDTO.getItemId())
            .orElseThrow(() -> new IllegalArgumentException(
                "Вещь с ID " + slotDTO.getItemId() + " не
найдена"));

        if (item.getCategory() != slotDTO.getSlotType()) {
            throw new IllegalArgumentException(
                "Категория вещи '" + item.getCategory() +
                "' не соответствует типу слота '" +
slotDTO.getSlotType() + "'");
        }

        if (!item.getUser().getId().equals(userId)) {
            throw new IllegalArgumentException("Вещь не принадлежит
текущему пользователю");
        }

        return item;
    }
}

```

ItemsViewModel.kt:

```
package com.myshef.myshef_app.presentation.viewmodel

import androidx.lifecycle.ViewModelScope
import com.myshef.myshef_app.data.local.entity.ItemLocal
import com.myshef.myshef_app.data.mapper.ItemMapper
import com.myshef.myshef_app.data.repository.ItemsRepository
import com.myshef.myshef_app.presentation.BaseViewModel
import com.myshef.myshef_app.presentation.item.ItemUpdates
import com.myshef.myshef_app.util.Resource
import com.myshef.myshef_app.util.Result
import kotlinx.coroutines.Job
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch

class ItemsViewModel(
    private val repository: ItemsRepository,
    private val userId: String
) : BaseViewModel() {

    private val _items =
    MutableStateFlow<Resource<List<ItemLocal>>>(Resource.Loading) =
    val items: StateFlow<Resource<List<ItemLocal>>> =
    _items.asStateFlow()

    private val _isLoading = MutableStateFlow(false)
    val isLoading: StateFlow<Boolean> = _isLoading.asStateFlow()

    private val _isRefreshing = MutableStateFlow(false)
    val isRefreshing: StateFlow<Boolean> =
    _isRefreshing.asStateFlow()

    private val _isSaving = MutableStateFlow(false)
    val isSaving: StateFlow<Boolean> = _isSaving.asStateFlow()

    private val _itemSaved = MutableStateFlow(false)
    val itemSaved: StateFlow<Boolean> = _itemSaved.asStateFlow()

    private val _errorMessage = MutableStateFlow<String?>(null)
    val errorMessage: StateFlow<String?> =
    _errorMessage.asStateFlow()

    private val allItems = mutableListOf<ItemLocal>()
    private var categoryFilter: String? = null
    private var searchQuery: String = ""
    private var loadJob: Job? = null

    fun loadItems() {
```

```

loadJob?.cancel()
loadJob = viewModelScope.launch {
    try {
        _isLoading.value = true
        _items.value = Resource.Loading
        _errorMessage.value = null

        repository.getItemsFlow(userId).collect { list ->
            allItems.clear()
            allItems.addAll(list)
            _items.value
Resource.Success(applyFilters(allItems))
            _isLoading.value = false
        }
    } catch (e: Exception) {
        _isLoading.value = false
        val message = getErrorMessage(e)
        _errorMessage.value = message
        _items.value = Resource.Error(message)
    }
}

fun createItem(
    name: String,
    description: String?,
    category: String,
    season: String?,
    imageUrl: String?
) {
    viewModelScope.launch {
        try {
            _isSaving.value = true
            _itemSaved.value = false
            _errorMessage.value = null

            val item = ItemLocal(
                id = ItemMapper.generateLocalId(),
                userId = userId,
                name = name,
                description = description,
                category = category,
                season = season,
                imageUrl = imageUrl,
                isDirty = true
            )

            when (val result = repository.createItem(item)) {
                is Result.Success -> {
                    _errorMessage.value = null
                    _itemSaved.value = true
                }
            }
        }
    }
}

```



```

        }

        is Result.Error -> _errorMessage.value =
result.message
    }
    } catch (e: Exception) {
        _errorMessage.value = getErrorMessage(e)
    } finally {
        _isSaving.value = false
    }
}

fun consumeItemSaved() {
    _itemSaved.value = false
}

fun loadItem(itemId: String): Flow<ItemLocal?> {
    return repository.getItemFlow(itemId)
}

fun updateItem(itemId: String, updates: ItemUpdates) {
    viewModelScope.launch {
        try {
            _isSaving.value = true
            _itemSaved.value = false
            _errorMessage.value = null

            when (val result = repository.updateItem(itemId,
updates)) {
                is Result.Success -> {
                    _errorMessage.value = null
                    _itemSaved.value = true
                }

                is Result.Error -> _errorMessage.value =
result.message
            }
        } catch (e: Exception) {
            _errorMessage.value = getErrorMessage(e)
        } finally {
            _isSaving.value = false
        }
    }
}

fun deleteItem(itemId: String) {
    viewModelScope.launch {
        try {
            _isLoading.value = true
            _errorMessage.value = null

```

```

        when (val result = repository.deleteItem(itemId)) {
            is Result.Success -> _errorMessage.value = null
            is Result.Error -> _errorMessage.value =
result.message
        }
    } catch (e: Exception) {
        _errorMessage.value = getErrorMessage(e)
    } finally {
        _isLoading.value = false
    }
}

fun syncItems() {
    viewModelScope.launch {
        try {
            _isRefreshing.value = true
            _errorMessage.value = null

            when (val result =
repository.syncItemsWithServer(userId)) {
                is Result.Success -> _errorMessage.value = null
                is Result.Error -> _errorMessage.value =
result.message
            }
        } catch (e: Exception) {
            _errorMessage.value = getErrorMessage(e)
        } finally {
            _isRefreshing.value = false
        }
    }
}

fun filterByCategory(category: String?) {
    categoryFilter = category?.takeIf { it.isNotBlank() }
    publishFilteredItems()
}

fun search(query: String) {
    searchQuery = query.trim()
    publishFilteredItems()
}

fun clearError() {
    _errorMessage.value = null
}

private fun publishFilteredItems() {
    if (allItems.isEmpty() && _items.value is Resource.Loading)
return

```

```

        _items.value = Resource.Success(applyFilters(allItems))
    }

    private fun applyFilters(source: List<ItemLocal>):
List<ItemLocal> {
        return source
            .asSequence()
            .filter { item ->
                categoryFilter == null ||
item.category.equals(categoryFilter, ignoreCase = true)
            }
            .filter { item ->
                searchQuery.isBlank() ||
                item.name.contains(searchQuery, ignoreCase =
true) ||
                item.description?.contains(searchQuery,
ignoreCase = true) == true
            }
            .toList()
    }

    override fun onCleared() {
        loadJob?.cancel()
        super.onCleared()
    }
}

```

V1__init_database.sql (только DDL):

```
-- Категории вещей
CREATE TYPE category_enum AS ENUM (
    'HEADWEAR',    -- Головной убор
    'TOP',          -- Верх
    'BOTTOM',       -- Низ
    'SHOES',        -- Обувь
    'OUTERWEAR',    -- Верхняя одежда
    'ACCESSORIES'   -- Аксессуары
);

-- Сезонность
CREATE TYPE season_enum AS ENUM (
    'WINTER',
    'SPRING',
    'SUMMER',
    'AUTUMN',
    'ALL_SEASONS'
);

-- Таблица пользователей
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    display_name VARCHAR(100),
    avatar_url VARCHAR(2048),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Таблица настроек пользователя
CREATE TABLE user_settings (
    user_id UUID PRIMARY KEY, -- PK является также FK
    theme VARCHAR(20) DEFAULT 'LIGHT',
    language VARCHAR(10) DEFAULT 'RU',
    notifications_enabled BOOLEAN DEFAULT TRUE,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_settings_user
        FOREIGN KEY (user_id)
        REFERENCES users(id)
        ON DELETE CASCADE -- Если удаляется юзер, удаляются и
настройки
);

-- Таблица вещей
CREATE TABLE items (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL,
    name VARCHAR(255) NOT NULL,
    description TEXT,
```

```

        image_url VARCHAR(2048),
        category category_enum NOT NULL,
        season season_enum,
        created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
        CONSTRAINT fk_items_user
            FOREIGN KEY (user_id)
            REFERENCES users(id)
            ON DELETE CASCADE -- При удалении юзера удаляются все его
вещи
    );

-- Таблица образов (Outfits)
CREATE TABLE outfits (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL,
    name VARCHAR(255) NOT NULL,
    description TEXT,
    season season_enum,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_outfits_user
        FOREIGN KEY (user_id)
        REFERENCES users(id)
        ON DELETE CASCADE -- При удалении юзера удаляются все его
образы
);

-- Таблица слотов образа (Outfit Slots)
CREATE TABLE outfit_slots (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    outfit_id UUID NOT NULL,
    item_id UUID, -- Слот может быть пустым (шаблон образа)
    slot_type category_enum NOT NULL,
    CONSTRAINT fk_slots_outfit
        FOREIGN KEY (outfit_id)
        REFERENCES outfits(id)
        ON DELETE CASCADE, -- При удалении образа удаляются его
слоты
    CONSTRAINT fk_slots_item
        FOREIGN KEY (item_id)
        REFERENCES items(id)
        ON DELETE SET NULL -- При удалении вещи слот не удаляется, а
обнуляется.
);

-- Образ сохраняется как структура, но
без конкретной вещи.
);

```

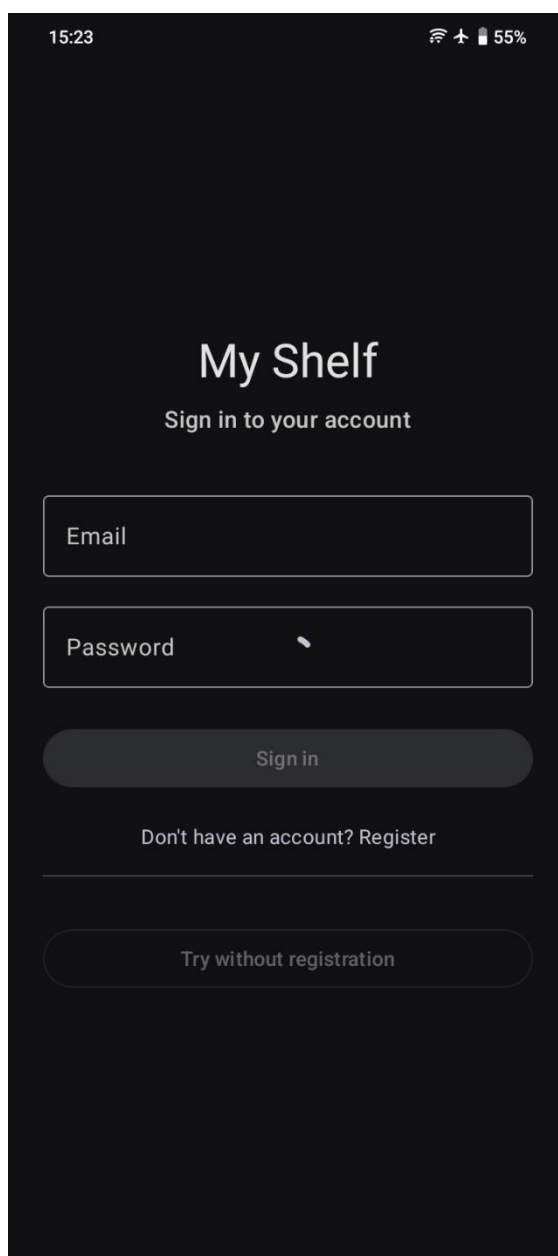


Рисунок 9. Экран аутентификации

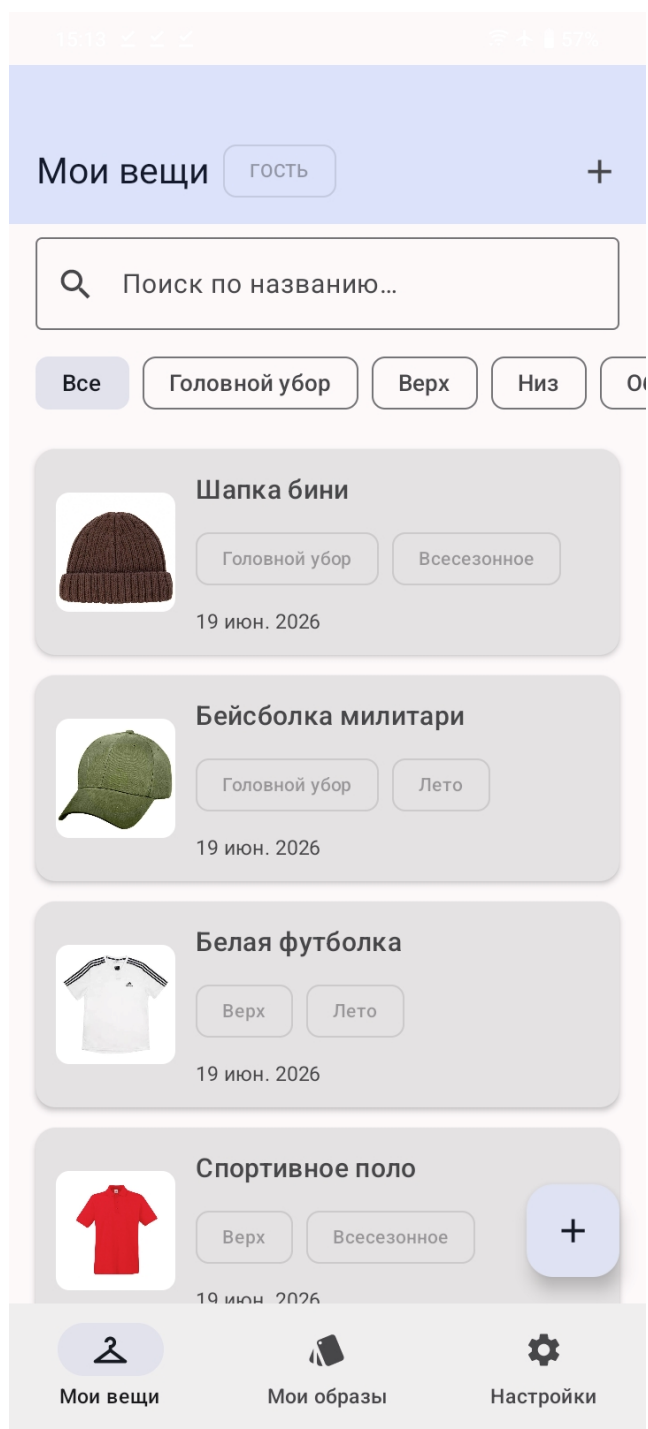


Рисунок 10. Экран "Мои вещи"

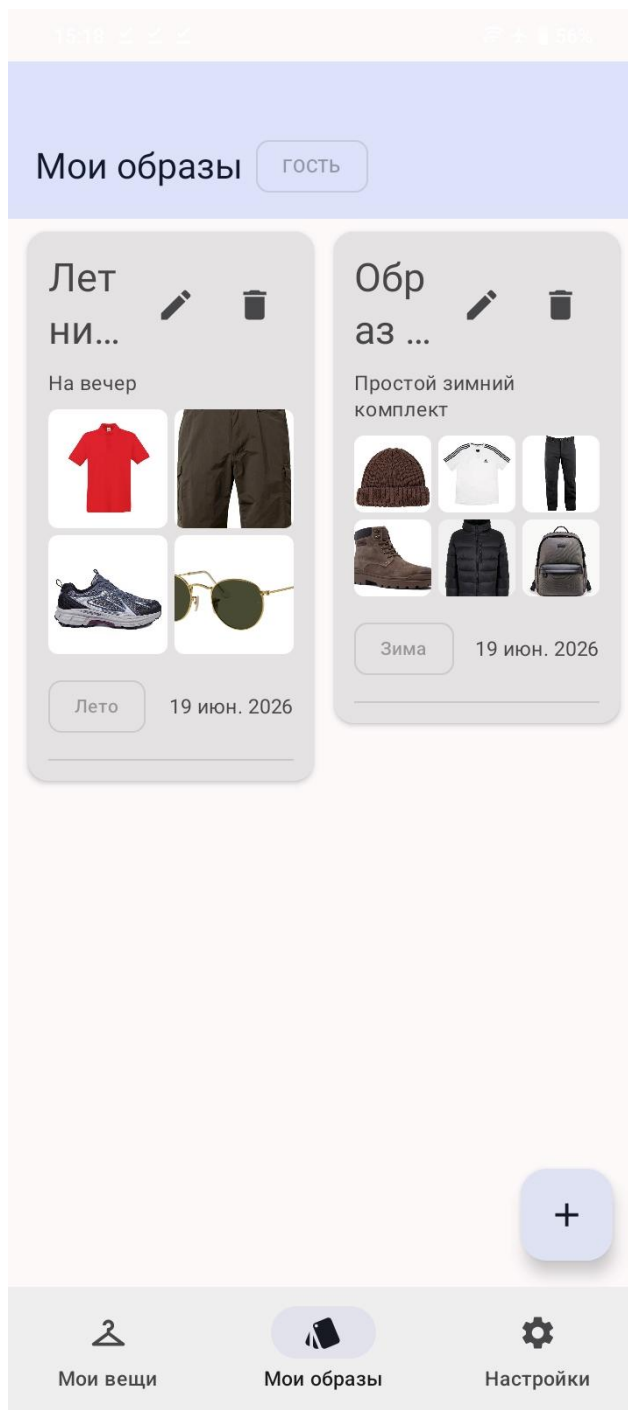


Рисунок 11. Экран "Мои образы"

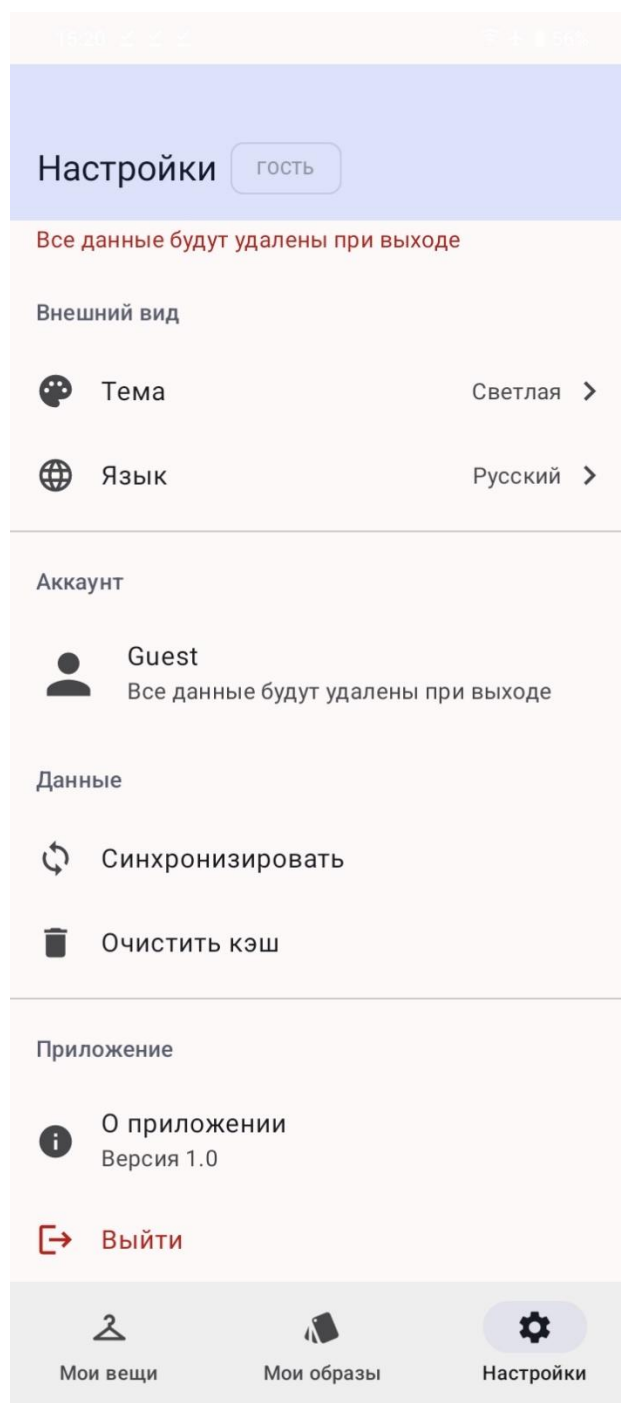


Рисунок 12. Экран "Настройки"

Приложение В. Результаты тестирования

my-shelf-server

my-shelf-server

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.myshef.wardrobe.entity		62 %		50 %	66	142	28	77	65	141	1	13
com.myshef.wardrobe.security		0 %		0 %	13	13	38	38	8	8	2	2
com.myshef.wardrobe.dto		80 %		n/a	15	62	30	124	15	62	0	7
com.myshef.wardrobe.config		0 %		n/a	7	7	13	13	7	7	1	1
com.myshef.wardrobe.controller		70 %		n/a	5	17	13	43	5	17	1	3
com.myshef.wardrobe.service		96 %		93 %	3	47	8	188	1	32	0	3
com.myshef.wardrobe		37 %		n/a	1	2	2	3	1	2	0	1
Total	650 of 2 207	70 %	13 of 42	69 %	110	290	132	486	102	269	5	30

Рисунок 13. Отчет JaCoCo о покрытии тестами

```
[INFO] Tests run: 69, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.11:report (report) @ my-shelf-server ---
[INFO] Loading execution data file C:\Users\boror\OneDrive\Desktop\myShelf\my-shelf-server\target\jacoco.exec
[INFO] Analyzed bundle 'my-shelf-server' with 39 classes
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 28.419 s
[INFO] Finished at: 2026-06-17T21:24:02+03:00
[INFO] -----
```

Рисунок 14. Отчет о тестировании сервера Spring Boot в консоли GigaIDE